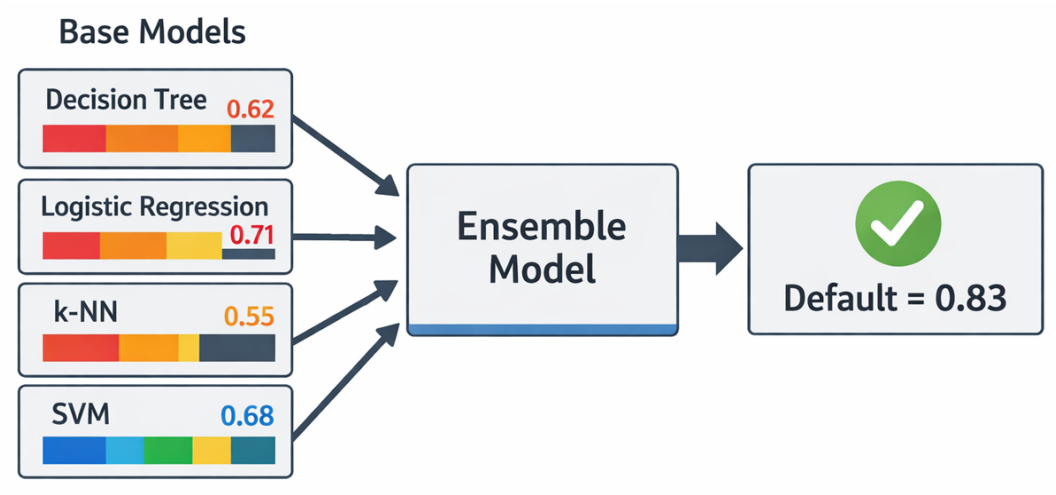
Chapter 14: Ensemble Methods

14.1Introduction



In a prior chapter, we built classification models using individual algorithms such as logistic regression, decision trees, k-nearest neighbors, and support vector machines. Each of these models can perform well, but each also has systematic weaknesses that limit how far its performance can be pushed.

Ensemble methods address this limitation by combining the predictions of many models into a single predictor. Rather than relying on one imperfect model, we allow multiple models to contribute evidence toward each prediction.

The core idea is simple: if different models make different mistakes, their errors can partially cancel out when combined. When designed carefully, the resulting ensemble is often more accurate, more stable, and better calibrated than any single model alone.

Ensemble methods address model weaknesses through two primary mechanisms: variance reduction and bias reduction. Variance reduction occurs when averaging multiple models smooths out the instability of individual predictions. Bias reduction occurs when sequential models focus on correcting the errors of previous models. Different ensemble approaches emphasize these mechanisms to different degrees:

Table 14.1
How ensemble methods address bias and variance

Ensemble method	Primary mechanism

Ensemble method	Primary mechanism
Bagging / Random Forests	Primarily reduce variance by averaging many decorrelated models trained on bootstrap samples
Boosting (AdaBoost, Gradient Boosting, XGBoost)	Reduce both bias and variance by sequentially correcting errors and averaging many weak learners
Stacking	Can reduce both bias and variance, depending on the diversity and quality of base learners

By the end of this chapter, you should be able to:

- Explain why ensembles often outperform single models.
- Describe the difference between variance reduction and bias reduction.
- Train and evaluate several common ensemble algorithms using scikit-learn.
- Interpret ensemble performance using accuracy, log loss, and class-level metrics.

To understand why ensembles help, it is useful to first examine the typical weaknesses of single-model approaches. Different algorithms fail in different ways, which creates opportunities for improvement through combination.

Table 14.2
Common weaknesses of individual classification models

Model type	Main strength	Typical weakness
Logistic regression	Stable, interpretable, well-calibrated probabilities	High bias; cannot model complex nonlinear boundaries
Decision trees	Flexible, nonlinear, easy to visualize	High variance; sensitive to small data changes
k-NN	Simple, nonparametric	Sensitive to noise and feature scaling
SVM	Strong margins, good accuracy	Computationally expensive and difficult to tune

Decision trees represent the classic high-variance case: two trees trained on slightly different samples can produce very different structures and predictions. This instability makes trees ideal candidates for ensemble techniques that average across many trees.

Linear models such as logistic regression represent the opposite extreme: they are stable but often too simple to capture real-world complexity. In this case, ensembles can help by combining multiple weak linear boundaries into a more expressive decision function.

Many algorithms are also sensitive to noise, outliers, or small sample fluctuations, which further increases prediction error. Ensembles reduce this sensitivity by spreading risk across multiple learned representations.

This logic closely mirrors the **wisdom of crowds** principle: while individuals may be unreliable, the average of many diverse opinions is often surprisingly accurate. Ensemble learning formalizes this idea in a statistical framework.

From a theoretical perspective, ensembles improve performance by altering the balance between *bias* and *variance*. Some methods primarily reduce variance, others reduce bias, and some attempt to control both simultaneously.

Throughout this chapter, we will continue using the Lending Club dataset introduced earlier to illustrate ensemble classification techniques. This continuity allows direct comparison between single models and their ensemble counterparts.

Although our examples focus on classification, nearly every ensemble method discussed also has a regression version and is widely used for predicting continuous outcomes such as prices, demand, and financial risk.

14.2Baseline Models

Data Preparation

Before we build baseline models or ensembles, we will apply a small set of data preparation steps that remain consistent across the entire chapter.

This consistency matters because ensemble methods compare many models side-by-side, and we want differences in performance to come from the algorithms (not from changing preprocessing choices).

Drop columns that are not useful for modeling

We will drop *loan_status_numeric* (it can leak information or duplicate the label definition) and remove text-heavy identifier-style fields (*emp_title* and *title*) that are not used in this chapter.

Convert issue date into a numeric recency feature

Because most machine learning algorithms do not use raw date strings directly, we convert *issue_d* into a numeric feature representing how many days ago the loan was issued relative to the most recent loan in the dataset.

Handle missing values

The columns *mths_since_last_delinq* and *mths_since_last_record* are missing when a borrower has never had a delinquency or a public record; these missing values are meaningful, not random.

A good approach is to (1) add a binary indicator for whether the borrower has ever had a delinquency or record and (2) fill missing values with a "large" number (such as the observed maximum plus one) to represent "not observed in history."

For the rest of the predictors, we recommend imputing numeric columns with the median and categorical columns with the most frequent category (or an explicit *Unknown* label), which we will implement inside the preprocessing pipeline so it is learned only from the training data.

Other cleaning opportunities in this dataset

- Convert *term* from strings like "36 months" into an integer number of months.
- Convert *emp_length* into an approximate numeric value in years (and keep an *Unknown* category if it is missing).
- Treat remaining string fields as categorical predictors and rely on one-hot encoding rather than manual recoding.

```
import pandas as pd
import numpy as np

df = pd.read_csv(&quot;lc_small.csv&quot;)
drop_cols = [&quot;loan_status_numeric&quot;, &quot;emp_title&quot;, &quot;title&quot;]
drop_cols = [c for c in drop_cols if c in df.columns]
df = df.drop(columns=drop_cols)
df[&quot;issue_d&quot;] = pd.to_datetime(df[&quot;issue_d&quot;], errors=&quot;coerce&quot;)
max_issue_date = df[&quot;issue_d&quot;].max()
df[&quot;issue_age_days&quot;] = (max_issue_date - df[&quot;issue_d&quot;]).dt.days
df = df.drop(columns=[&quot;issue_d&quot;])

if &quot;term&quot; in df.columns:
  df[&quot;term&quot;] = df[&quot;term&quot;].astype(str).str.strip().str.extract(r&quot;
(\d+)&quot;).astype(float)

if &quot;emp_length&quot; in df.columns:
  emp = df[&quot;emp_length&quot;].astype(str).str.strip()
  emp = emp.replace({&quot;nan&quot;: np.nan, &quot;None&quot;: np.nan})

  emp = emp.replace({
    &quot;10+ years&quot;: &quot;10&quot;,
    &quot;&lt; 1 year&quot;: &quot;0&quot;
  })

  emp = emp.str.extract(r&quot;(\d+)&quot;)[0]
  df[&quot;emp_length_years&quot;] = pd.to_numeric(emp, errors=&quot;coerce&quot;)
  df = df.drop(columns=[&quot;emp_length&quot;])

for col in [&quot;mths_since_last_delinq&quot;, &quot;mths_since_last_record&quot;]:
  if col in df.columns:
    ind_col = col + &quot;_missing&quot;
    df[ind_col] = df[col].isna().astype(int)
    max_val = df[col].max(skipna=True)
    fill_val = (max_val + 1) if pd.notna(max_val) else 0
    df[col] = df[col].fillna(fill_val)
```

After running this cell, you should keep using the updated *df* object throughout the rest of Chapter 14 so that every baseline model and ensemble is trained on the same cleaned inputs.

Why we revisit single models

Before we build ensembles, we need a clear baseline for what “good performance” looks like using single models. This section recreates several models from the prior chapter so we can compare them fairly against bagging and boosting later.

Ensembles usually improve performance by reducing common weaknesses in single models, such as overfitting in trees or limited flexibility in linear models. To see that improvement clearly, we must keep the dataset, features, preprocessing, and train/test split identical across models.

Models included in the baseline comparison

We will rebuild four familiar classifiers: logistic regression, a shallow decision tree, k-nearest neighbors (k-NN), and Naive Bayes. Each model represents a different modeling philosophy, which helps motivate why combining models can work so well.

- **Logistic regression** (one-vs-rest): a strong linear baseline with interpretable coefficients and typically reasonable probability estimates.
- **Decision tree** (shallow depth): an interpretable nonlinear baseline that can capture interactions but may overfit as depth increases.
- **k-NN**: a distance-based approach that can model complex boundaries but can be sensitive to scaling and noisy features.
- **Naive Bayes**: a fast probabilistic model that makes strong independence assumptions and can struggle when features are correlated.

Step 1: Reload the dataset and recreate train/test objects

The code below rebuilds *X*, *y*, the train/test split, and the preprocessing pipeline. It is written to be easy to adapt to your exact Lending Club file and column names.

Important: if your prior chapter already created *df*, *X*, and *y*, you can reuse those and skip the loading and target-recoding parts. The key requirement is that you keep the same features and the same label definition so that later ensemble comparisons are valid.

```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler, FunctionTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

bad_statuses = {&quot;Charged Off&quot;, &quot;Default&quot;}
df[&quot;loan_good&quot;] = (~df[&quot;loan_status&quot;].isin(bad_statuses)).astype(int)
y = df[&quot;loan_good&quot;].copy()
X = df.drop(columns=[&quot;loan_status&quot;, &quot;loan_good&quot;]).copy()

X_train, X_test, y_train, y_test = train_test_split(
  X, y,
  test_size=0.20,
  random_state=27,
  stratify=y
```

```
)

cat_cols = X_train.select_dtypes(include=[&quot;object&quot;, &quot;category&quot;,
&quot;bool&quot;]).columns.tolist()
num_cols = X_train.select_dtypes(include=[&quot;number&quot;]).columns.tolist()

numeric_pipe = Pipeline(steps=[
  (&quot;impute&quot;, SimpleImputer(strategy=&quot;median&quot;)),
  (&quot;scale&quot;, StandardScaler())
])

categorical_pipe = Pipeline(steps=[
  (&quot;impute&quot;, SimpleImputer(strategy=&quot;most_frequent&quot;)),
  (&quot;onehot&quot;, OneHotEncoder(handle_unknown=&quot;ignore&quot;))
])

preprocessor = ColumnTransformer(
  transformers=[
    (&quot;num&quot;, numeric_pipe, num_cols),
    (&quot;cat&quot;, categorical_pipe, cat_cols)
  ],
  remainder=&quot;drop&quot;,
  sparse_threshold=0.3
)
```

Because one-hot encoding often creates many columns, the preprocessor may output a sparse matrix. That is efficient for most models, but a few algorithms (notably *GaussianNB*) require a dense matrix, so we will handle that explicitly when we build the Naive Bayes pipeline.

Step 2: Rebuild baseline models

Next, we define four pipelines that all share the same preprocessing step. This ensures that any performance differences come from the modeling algorithm rather than from inconsistent data preparation.

```
from sklearn.linear_model import LogisticRegression
from sklearn.multiclass import OneVsRestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB

# Logistic regression (OvR)
lr_base = LogisticRegression(
  solver=&quot;liblinear&quot;,
  max_iter=2000,
  random_state=27
)

model_lr = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;lr&quot;, OneVsRestClassifier(lr_base))
])

# Decision tree baseline (shallow)
model_tree3 = Pipeline(steps=[
```

```
  (&quot;prep&quot;, preprocessor),
  (&quot;tree&quot;, DecisionTreeClassifier(
    max_depth=3,
    random_state=27
  ))
])

# k-NN baseline (k chosen for a reasonable starting point)
model_knn = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;knn&quot;, KNeighborsClassifier(n_neighbors=15))
])

# Naive Bayes (GaussianNB) requires dense features.
# Convert sparse matrix to dense using FunctionTransformer.
to_dense = FunctionTransformer(
  lambda X: X.toarray() if hasattr(X, &quot;toarray&quot;) else np.asarray(X),
  accept_sparse=True
)

model_nb = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;dense&quot;, to_dense),
  (&quot;nb&quot;, GaussianNB())
])
```

Step 3: Evaluate with accuracy, log loss, and the classification report

We evaluate each model using both threshold-based and probability-based metrics. Accuracy is easy to interpret, but log loss helps us judge probability quality, which matters in lending decisions that use risk thresholds.

We also include a classification report because class imbalance can make accuracy misleading. In credit risk, the positive class (defaults) is usually the minority class, so per-class precision and recall are often more operationally important than overall accuracy.

```
from sklearn.metrics import accuracy_score, log_loss, classification_report

def eval_model(name, model, X_train, y_train, X_test, y_test):
  model.fit(X_train, y_train)
  y_pred = model.predict(X_test)
  acc = accuracy_score(y_test, y_pred)
  # Log loss requires predicted probabilities.
  # Some models provide predict_proba; if not, we skip log loss.
  ll = None

  if hasattr(model, &quot;predict_proba&quot;):
    y_prob = model.predict_proba(X_test)
    ll = log_loss(y_test, y_prob)

  print(&quot;\n&quot; + &quot;=&quot; * 70)
  print(name)
  print(&quot;Accuracy:&quot;, round(acc, 4))
```

```
    if ll is not None:
      print(&quot;Log loss:&quot;, round(ll, 4))
    else:
      print(&quot;Log loss: n/a (no predict_proba)&quot;)

    print(&quot;\nClassification report:&quot;)
    print(classification_report(y_test, y_pred, digits=3, zero_division=0))

    return {
      &quot;model&quot;: name,
      &quot;accuracy&quot;: acc,
      &quot;log_loss&quot;: ll
    }

  results = []
  results.append(eval_model(&quot;Logistic regression (OvR)&quot;, model_lr, X_train, y_train,
X_test, y_test))
  results.append(eval_model(&quot;Decision tree (depth=3)&quot;, model_tree3, X_train, y_train,
X_test, y_test))
  results.append(eval_model(&quot;k-NN (k=15)&quot;, model_knn, X_train, y_train, X_test,
y_test))
  results.append(eval_model(&quot;Naive Bayes (GaussianNB)&quot;, model_nb, X_train, y_train,
X_test, y_test))
  results_df = pd.DataFrame(results)
  results_df = results_df.sort_values(by=[&quot;log_loss&quot;, &quot;accuracy&quot;],
ascending=[True, False])
  results_df


  # Output:
  # Decision tree (depth=3)
  # Accuracy: 0.9342
  # Log loss: 0.2156
  #
  # Classification report:
  #               precision    recall  f1-score   support
  #
  #           0      0.875     0.272     0.415       180
  #           1      0.936     0.996     0.965      1916
  #          accuracy                          0.934      2096
  #         macro avg      0.905     0.634     0.690      2096
  #      weighted avg      0.931     0.934     0.918      2096
  #  ======================================================================
  #  k-NN (k=15)
  #  Accuracy: 0.916
  #  Log loss: 0.5692
  #  Classification report:
  #                precision    recall  f1-score   support
  #            0      0.611     0.061     0.111       180
  #            1      0.919     0.996     0.956      1916
  #      accuracy                           0.916      2096
  #    macro avg      0.765     0.529     0.534      2096
  #  weighted avg      0.892     0.916     0.883      2096
  #  ======================================================================
  #  Naive Bayes (GaussianNB)
  #  Accuracy: 0.2238
  #  Log loss: 27.9785
  #  Classification report:
  #                precision    recall  f1-score   support
  #            0      0.087     0.844     0.157       180
  #            1      0.919     0.165     0.280      1916
  #      accuracy                           0.224      2096
  #    macro avg      0.503     0.505     0.219      2096
  #  weighted avg      0.847     0.224     0.270      2096
```

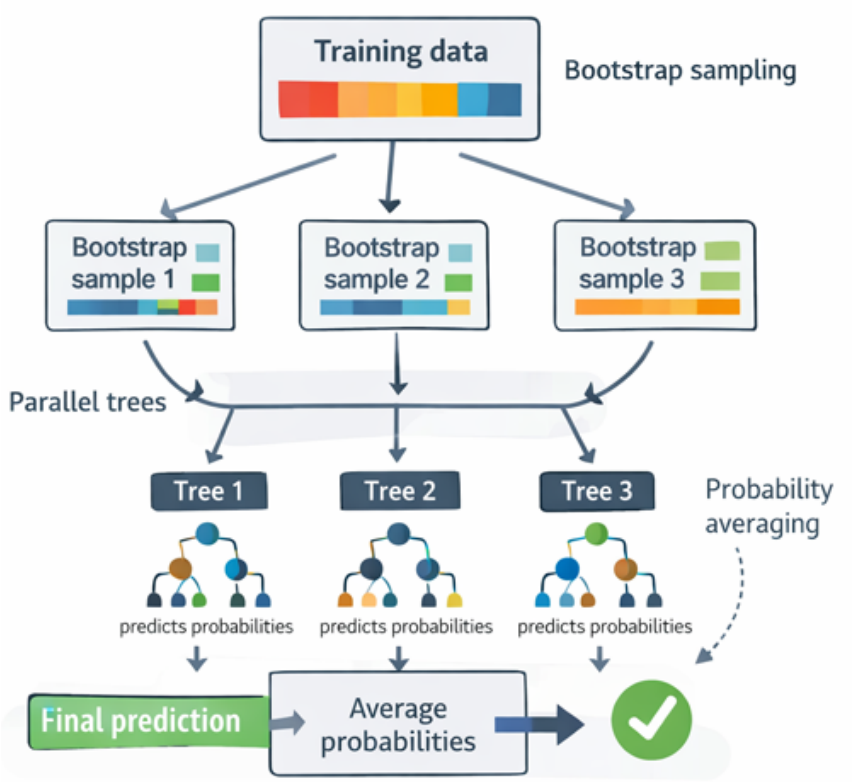
	model	accuracy	log_loss
0	Logistic regression (OvR)	0.954198	0.129619
1	Decision tree (depth=3)	0.934160	0.215635
2	k-NN (k=15)	0.916031	0.569203
3	Naive Bayes (GaussianNB)	0.223760	27.978542

Deliverable: A small comparison table of single models

Your comparison table now provides a concrete baseline for the rest of the chapter. In this dataset, logistic regression achieved the strongest overall performance among the single models, with an accuracy of 0.954 and a log loss of 0.130. The decision tree (depth = 3) followed with an accuracy of 0.934 and log loss of 0.216, while k-NN reached 0.916 accuracy but with substantially worse probability quality (log loss = 0.569). Naive Bayes performed very poorly, with only 0.224 accuracy and extremely high log loss (27.98), indicating badly miscalibrated probabilities.

These results also highlight why accuracy alone can be misleading. Both the decision tree and k-NN models achieved high overall accuracy, yet their classification reports show very weak recall for the minority class (only 0.272 for the tree and 0.061 for k-NN). In contrast, logistic regression maintained strong performance on both classes while also producing the most reliable probability estimates. This table will serve as the benchmark for evaluating whether ensemble methods can simultaneously improve minority-class detection and probability quality, not just headline accuracy.

14.3Bagging: Bootstrap Aggregation



Bagging, short for **bootstrap aggregation**, is one of the simplest and most powerful ensemble techniques. Its primary goal is to reduce model variance by averaging many noisy but flexible models instead of relying on a single unstable one.

Decision trees are especially good candidates for bagging because small changes in training data can lead to very different tree structures. This instability makes trees high-variance models, which means they often overfit even when they achieve high training accuracy.

Key idea

Rather than training one tree on the full dataset, bagging trains many trees on slightly different versions of the training data created by random resampling. Each tree makes its own prediction, and the ensemble combines those predictions into a single result.

- **Bootstrap sampling:** each model is trained on a random sample drawn with replacement from the training set.

- **Independent models:** trees are trained in parallel, not sequentially.
- **Aggregation:** predicted probabilities are averaged across all trees.

Because the individual trees make different mistakes, averaging their predictions tends to cancel out extreme errors. The result is a model that is more stable and usually generalizes better than any single tree.

Algorithm overview

Table 14.3
Bagging classification procedure

Step	Description
1	Draw a bootstrap sample from the training data by randomly sampling observations *with replacement*, creating a dataset the same size as the original but with repeated and omitted cases.
2	Train a decision tree on this bootstrap sample using the same feature set and hyperparameters as the base model.
3	Repeat steps 1–2 many times (for example, 50–500 trees) to create a collection of diverse models trained on slightly different data.
4	For each new observation, collect the predicted *class probabilities* from every tree in the ensemble.
5	Average these probabilities across all trees and select the class with the highest average probability as the final prediction.

Bagging does not directly reduce bias because each tree is still a flexible model that can fit complex patterns. Its strength lies in reducing variance by stabilizing predictions across many slightly different models.

Bagging in scikit-learn

Scikit-learn implements bootstrap aggregation through the *BaggingClassifier* class. We reuse the same preprocessing pipeline and train/test split as the baseline models so that any performance differences can be attributed to bagging rather than changes in data preparation.

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

base_tree = DecisionTreeClassifier(
  max_depth=3,
  random_state=27
)

model_bagging = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;bag&quot;, BaggingClassifier(
      estimator=base_tree,
      n_estimators=100,
      bootstrap=True,
      n_jobs=-1,
      random_state=27
  ))
])
```

We intentionally use the same tree depth as the single-tree baseline to isolate the effect of aggregation itself. Any improvement in accuracy or log loss therefore comes from averaging many unstable models rather than increasing the complexity of each individual tree.

The most important bagging parameters control how many models are trained, how the data are resampled, and how predictions are combined. Understanding these settings helps you reason about the bias–variance tradeoff and computational cost.

Table 14.4
Key BaggingClassifier parameters

Parameter	Meaning
estimator	The base model trained on each bootstrap sample (here, a shallow decision tree). This model should usually be high-variance so that averaging is beneficial.
n_estimators	The number of bootstrap models to train. Larger values reduce variance but increase training time; 50–300 is common in practice.
bootstrap	Whether training sets are created by sampling with replacement. Setting this to true enables classical bootstrap aggregation.
n_jobs	The number of CPU cores used for training. A value of −1 uses all available cores to train trees in parallel.
random_state	Controls the randomness of bootstrap sampling for reproducibility.

In classification, bagging averages the predicted class probabilities from all trees and then selects the class with the highest average probability. This probability averaging is the main reason bagging often improves log loss more reliably than accuracy.

Evaluate and compare to a single tree

```
bagging_results = eval_model(
  &quot;Bagging (100 trees, depth=3)&quot;,
  model_bagging,
  X_train,
  y_train,
  X_test,
  y_test
)

results_df = pd.concat(
  [results_df, pd.DataFrame([bagging_results])],
  ignore_index=True
).sort_values(by=[&quot;log_loss&quot;, &quot;accuracy&quot;], ascending=[True, False])
results_df


# Output:
# ====================================================================
# Bagging (100 trees, depth=3)
# Accuracy: 0.9356
# Log loss: 0.1949
#  Classification report:
#                precision    recall  f1-score   support
#                0      0.979     0.256     0.405       180
#                1      0.935     0.999     0.966      1916
#          accuracy                          0.936      2096
#        macro avg      0.957     0.628     0.686      2096
#      weighted avg      0.938     0.936     0.918      2096
```

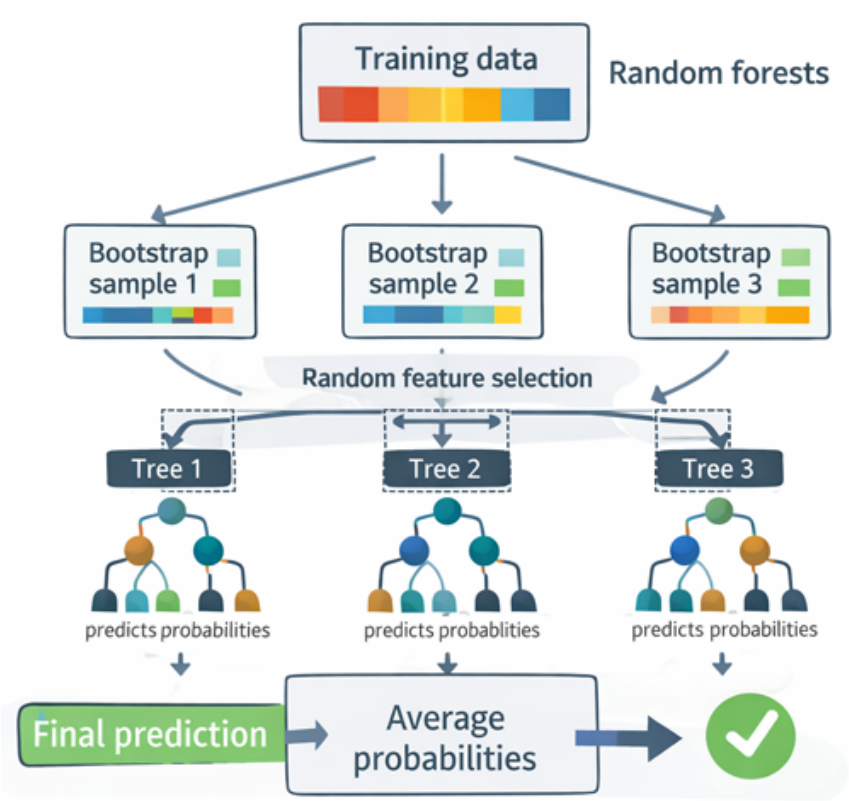
	model	accuracy	log_loss
0	Logistic regression (OvR)	0.954198	0.129619
4	Bagging (100 trees, depth=3)	0.935592	0.194865
1	Decision tree (depth=3)	0.934160	0.215635
2	k-NN (k=15)	0.916031	0.569203
3	Naive Bayes (GaussianNB)	0.223760	27.978542

In this experiment, bagging improves probability quality more than raw classification accuracy. The single decision tree achieved an accuracy of *0.9342* with log loss *0.2156*, while the bagged ensemble of 100 trees reached a similar accuracy of *0.9356* but a noticeably lower log loss of *0.1949*. This confirms that averaging predictions stabilizes

probability estimates even when overall accuracy changes only marginally. The classification report also shows that recall for the minority class (class 0) remains low at *0.256*, illustrating that bagging primarily reduces variance rather than fundamentally changing class separation.

In the next section, we extend this idea further by allowing trees to influence each other during training using *boosting*, which explicitly targets both variance and bias rather than averaging independent models.

14.4Random Forests



Bagging reduces variance by averaging many high-variance models trained on resampled data. Random forests extend this idea further by also injecting randomness into the feature selection process at each split.

This additional randomness decorrelates the trees, making the average prediction more stable and often more accurate than bagging alone. In practice, random forests are one

of the strongest general-purpose classification algorithms available.

Why does feature randomness improve ensemble performance? When all trees in a bagging ensemble consider the same set of features at each split, they tend to identify the same strong predictors and make similar decisions. This correlation means that when one tree makes an error, other trees are likely to make the same error, reducing the benefit of averaging. By randomly restricting which features each tree can consider at each split, random forests force trees to explore different decision boundaries. Some trees may focus on one set of features while others focus on different features, creating diversity in their predictions. When these decorrelated trees are averaged, their different mistakes partially cancel out, leading to more stable and often more accurate predictions than bagging alone.

Key ideas behind random forests

- Each tree is trained on a bootstrap sample of the training data.
- At every split, only a random subset of features is considered.
- This feature randomness reduces correlation between trees.
- Predicted class probabilities are averaged across all trees.

If all trees see the same strong predictors at every split, they tend to make similar mistakes. Random feature selection forces trees to explore different decision boundaries, improving generalization.

Training a random forest classifier

We now train a random forest using the same preprocessing pipeline and data split as the baseline and bagging models. This allows a clean comparison of performance and probability quality.

```
from sklearn.ensemble import RandomForestClassifier

rf_model = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;rf&quot;, RandomForestClassifier(
      n_estimators=200,
```

```
        max_depth=None,
        min_samples_leaf=1,
        random_state=27,
        n_jobs=-1
    ))
])

rf_model.fit(X_train, y_train)
```

The random forest model is controlled by several key hyperparameters. *n_estimators* specifies how many decision trees are trained in parallel; larger values usually improve stability and accuracy but increase training time. *max_depth* limits how deep each tree can grow, with *None* allowing fully grown trees that capture complex interactions. *min_samples_leaf* sets the minimum number of observations required in a leaf node and acts as a regularization control to prevent overly specific splits. Finally, *n_jobs = -1* enables parallel training across all available CPU cores to significantly speed up fitting.

Unlike single decision trees, random forests intentionally introduce randomness through bootstrap sampling of rows and random selection of feature subsets at each split. Because this randomness is fundamental to how the algorithm reduces correlation between trees, the model does not rely on a fixed random seed for correctness or performance. Setting *random_state* is therefore optional and is used only to make results exactly reproducible for teaching or experimentation. By default, random forests grow deep trees and rely on averaging across many diverse models to control overfitting, which often produces strong accuracy but slightly less well-calibrated probabilities than simpler linear models.

Evaluating performance

We evaluate the model using accuracy, log loss, and a full classification report. Log loss is especially important for credit risk modeling because decisions depend on probability thresholds rather than only class labels. For example, a lender may approve loans with default probability below 5%, deny loans above 20%, and require manual review for probabilities in between. Accurate probability estimates are therefore critical for making these threshold-based decisions.

```
from sklearn.metrics import accuracy_score, log_loss, classification_report
```

```
y_rf_pred = rf_model.predict(X_test)
y_rf_prob = rf_model.predict_proba(X_test)
rf_acc = accuracy_score(y_test, y_rf_pred)
rf_ll = log_loss(y_test, y_rf_prob)

print(&quot;Random forest accuracy:&quot;, round(rf_acc, 4))
print(&quot;Random forest log loss:&quot;, round(rf_ll, 4))
print(&quot;\nClassification report:&quot;)
print(classification_report(y_test, y_rf_pred, digits=3, zero_division=0))


# Output:
#  Random forest accuracy: 0.9213
#  Random forest log loss: 0.1944
#  Classification report:
#                precision    recall  f1-score   support
#                0      1.000     0.083     0.154       180
#                1      0.921     1.000     0.959      1916
#           accuracy                          0.921      2096
#         macro avg      0.960     0.542     0.556      2096
#       weighted avg      0.928     0.921     0.890      2096
```

The random forest achieves a test-set accuracy of *0.9213* and a log loss of *0.1944*. While the accuracy is slightly lower than the single decision tree baseline (*0.9342*), the random forest achieves substantially better probability quality, with log loss improving from *0.2156* to *0.1944*. This confirms that averaging many decorrelated trees improves probability calibration and model stability, even when overall classification accuracy changes only marginally.

The log loss of *0.1944* indicates that the model's probability estimates are reasonably well calibrated, though not as strong as those produced by linear models in earlier sections. This reflects a common tradeoff with tree-based ensembles: higher predictive power but slightly less stable probability estimates.

Examining the classification report reveals an important limitation that is hidden by the high accuracy. For the minority class (class *0*), recall is only *0.083*, meaning that the model correctly identifies fewer than 9 percent of truly bad loans.

At the same time, precision for the minority class is *1.000*, which means that when the model does predict a loan as bad, it is almost always correct. This behavior indicates an extremely conservative decision boundary that avoids false positives but misses most risky borrowers.

For the majority class (class *1*), recall is *1.000* and precision is *0.921*, showing that the model nearly always labels good loans correctly. As a result, overall accuracy is dominated by performance on the majority class.

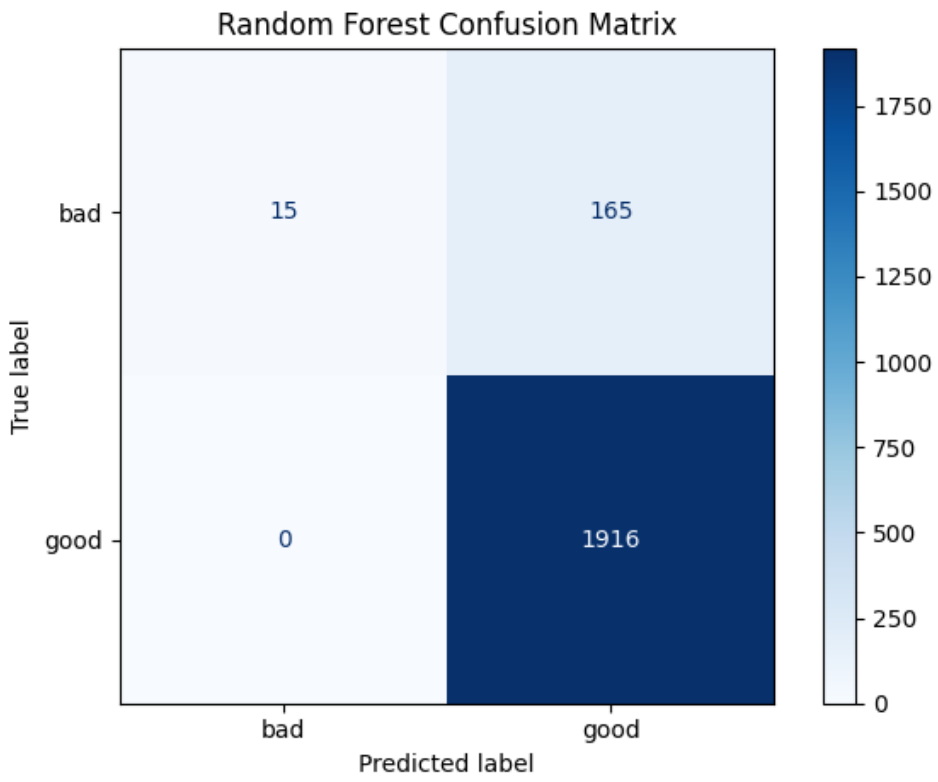
The macro-average recall of *0.542* highlights this imbalance, because it weights both classes equally and exposes how poorly the minority class is detected. In contrast, the weighted-average F1 score of *0.890* remains high because it is driven by the much larger good-loan population.

From a business perspective, this model would substantially reduce false alarms but would fail to flag most high-risk borrowers. Whether this tradeoff is acceptable depends on the cost of missed defaults versus the cost of rejecting safe customers.

Confusion matrix

The confusion matrix highlights how errors are distributed across the good and bad loan classes. This is critical for understanding whether the model sacrifices minority-class detection for higher overall accuracy.

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cm_rf = confusion_matrix(y_test, y_rf_pred)
plt.figure(figsize=(5, 4))
ConfusionMatrixDisplay(cm_rf, display_labels=[&quot;bad&quot;,
&quot;good&quot;]).plot(values_format=&quot;d&quot;, cmap='Blues')
plt.title(&quot;Random Forest Confusion Matrix&quot;)
plt.tight_layout()
plt.show()
```



The confusion matrix shows that the random forest correctly identifies *1,916* good loans but only *15* bad loans. At the same time, it misclassifies *165* bad loans as good and never incorrectly flags a good loan as bad.

This confirms that the model is extremely conservative when predicting the bad class. It avoids false alarms entirely, but at the cost of missing most risky borrowers, which explains the low recall observed earlier for the minority class.

Feature importance in random forests

Random forests provide built-in estimates of feature importance based on how much each variable reduces impurity across all trees. These values are known as mean decrease in impurity scores.

Although not causal, feature importance is often useful for model validation and business interpretation. It can reveal which borrower characteristics most strongly influence predicted default risk.

```
import pandas as pd
import numpy as np

rf_estimator = rf_model.named_steps[&quot;rf&quot;]
feature_names = rf_model.named_steps[&quot;prep&quot;].get_feature_names_out()

importances = pd.DataFrame({
  &quot;feature&quot;: feature_names,
  &quot;importance&quot;: rf_estimator.feature_importances_
}).sort_values(by=&quot;importance&quot;, ascending=False)
top_features = importances.head(15)
top_features
```

	feature	importance
5	num__total_rec_prncp	0.081252
23	num__issue_age_days	0.071423
4	num__total_pymnt	0.057010
3	num__installment	0.041361
2	num__int_rate	0.038875
6	num__total_rec_int	0.036799
0	num__loan_amnt	0.034229
22	num__dti	0.029949
19	num__tot_cur_bal	0.029786
17	num__revol_util	0.029541
8	num__annual_inc	0.028813
16	num__revol_bal	0.028418
21	num__total_rev_hi_lim	0.027798
20	num__total_acc	0.024357
7	num__total_rec_late_fee	0.022767

Feature importances from a random forest reflect which inputs most reduced impurity across the forest's splits. In other words, higher values indicate variables that the model

relied on more often and more strongly to separate loans that ended in good standing from loans that became bad.

The most important signals are concentrated in repayment and balance variables, such as *total_rec_prncp* (0.0813) and *total_pymnt* (0.0570). These features capture whether the borrower has already paid back principal and how much total payment has been received, which naturally aligns with the model's goal of distinguishing loans that are performing well from loans that are not.

Time and "age of loan" also matter: *issue_age_days* (0.0714) ranks second. This suggests the model uses loan seasoning as an important context variable, because the risk profile and repayment pattern of a newly issued loan can differ from one that has been active much longer.

Contract terms and pricing variables—such as *installment* (0.0414), *int_rate* (0.0389), and *loan_amnt* (0.0342)—are also influential. Operationally, this implies that payment burden and loan pricing are meaningful drivers of outcomes, and that risk segmentation can often be improved by considering affordability proxies like payment size and debt ratios.

Borrower leverage and revolving-credit behavior show up through *dti* (0.0299), *revol_util* (0.0295), *revol_bal* (0.0284), and *total_rev_hi_lim* (0.0278). These features support decisions about monitoring and intervention, because high utilization or high revolving balances can indicate liquidity strain that increases default risk.

Portfolio exposure and capacity indicators—such as *tot_cur_bal* (0.0298), *annual_inc* (0.0288), and *total_acc* (0.0244)—suggest the model considers broader financial context beyond the focal loan. In practice, these can inform policy rules or review thresholds (for example, requiring additional documentation or manual review when high utilization coincides with weaker income or thinner credit depth).

Two important cautions apply when interpreting this list. First, impurity-based importances can be biased toward continuous variables and toward features with many potential split points, so the ranking is best treated as a screening tool rather than a definitive causal claim.

Second, several top features (for example, *total_rec_prncp* and *total_pymnt*) may be "outcome-adjacent" because they reflect repayment progress after the loan is issued. If your business goal is early risk prediction at origination, you should exclude post-origination variables and recompute importances using only information available at decision time.

Additional caveats apply when interpreting feature importance rankings:

- **One-hot encoding can inflate importance of categorical features:** When a categorical variable with many levels is one-hot encoded, it creates multiple binary features. Random forests may split on several of these binary features, and the sum of their individual importances can make the original categorical variable appear more important than it actually is. To get a fair comparison, sum the importances of all one-hot encoded columns that belong to the same original categorical feature.
- **Impurity-based vs. permutation importance:** Random forests report *impurity-based* importance (mean decrease in impurity), which measures how much each feature reduces node impurity across all splits. This is fast to compute but can be biased toward high-cardinality features. *Permutation importance* (available in scikit-learn via *permutation_importance*) measures how much model performance degrades when a feature's values are randomly shuffled. Permutation importance is more reliable for comparing features fairly but requires refitting or scoring the model multiple times, making it computationally expensive.
- **When to trust vs. question feature importance rankings:** Trust rankings when (1) features are preprocessed consistently (no one-hot encoding artifacts), (2) you have sufficient data (importance estimates stabilize with more trees and more data), and (3) the ranking aligns with domain knowledge. Question rankings when (1) features are highly correlated (importance can be arbitrarily distributed among correlated features), (2) sample size is small (estimates are noisy), or (3) the ranking contradicts domain expertise (may indicate data quality issues or model problems).
- **Importance rankings are not consistent across model types:** Different algorithms can produce different importance rankings for the same features, even on the same data. A random forest may rank features differently from a gradient boosting model because each algorithm learns different tree structures and uses different splitting strategies. Logistic regression coefficients capture yet another perspective—linear

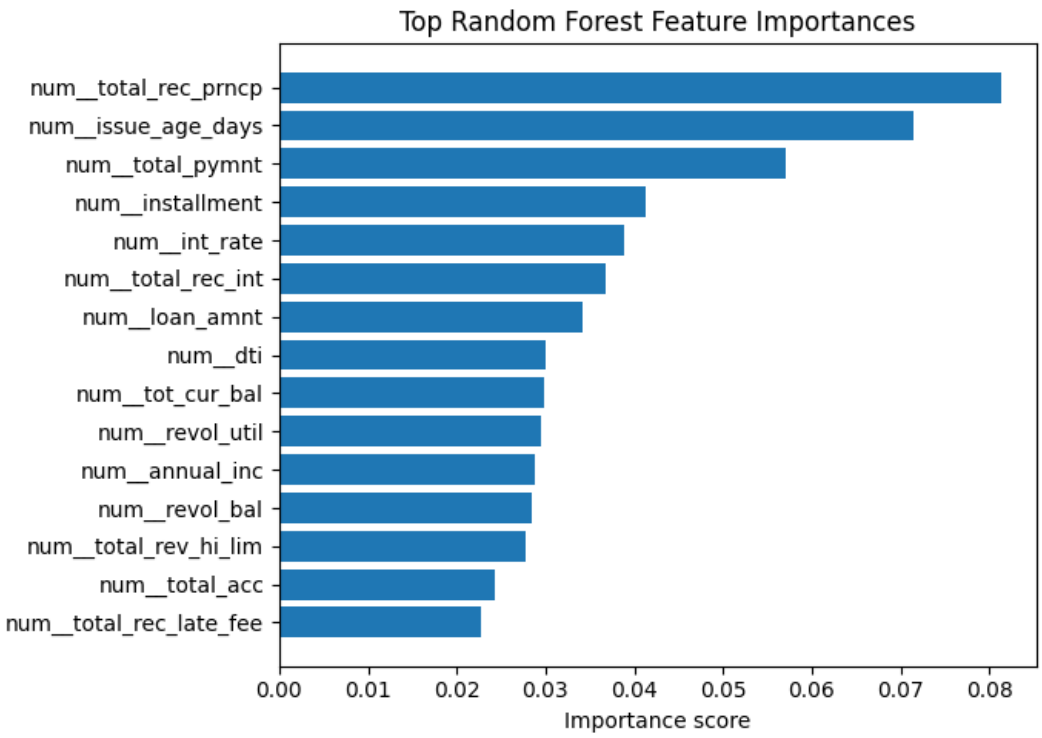
association strength—which may not align with tree-based importance at all. For this reason, treat any single model's importance ranking as one view of the data rather than a definitive ordering, and compare rankings across model types when making high-stakes feature decisions.

A practical decision workflow is to use this ranking to (1) identify a small set of high-signal variables for monitoring dashboards, (2) flag potential data leakage candidates to remove when building a true pre-origination model, and (3) guide feature engineering by creating clearer affordability and revolving-credit measures that capture the same underlying risk patterns.

Visualizing the most important features

Plotting the top features helps translate the model into business-relevant insights. Highly ranked variables often correspond to borrower risk, loan size, and repayment capacity.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(7, 5))
plt.barh(top_features[&quot;feature&quot;][::-1], top_features[&quot;importance&quot;][::-1])
plt.title(&quot;Top Random Forest Feature Importances&quot;)
plt.xlabel(&quot;Importance score&quot;)
plt.tight_layout()
plt.show()
```



Random forests typically outperform single trees in both accuracy and log loss because they reduce variance while preserving nonlinear decision boundaries. However, they are slower to train and less interpretable than individual trees.

In the next section, we move from parallel ensembles to sequential ensembles using boosting, where models actively learn from previous mistakes rather than operating independently.

Before we move on, let's see how random forest compares to our prior algorithm results.

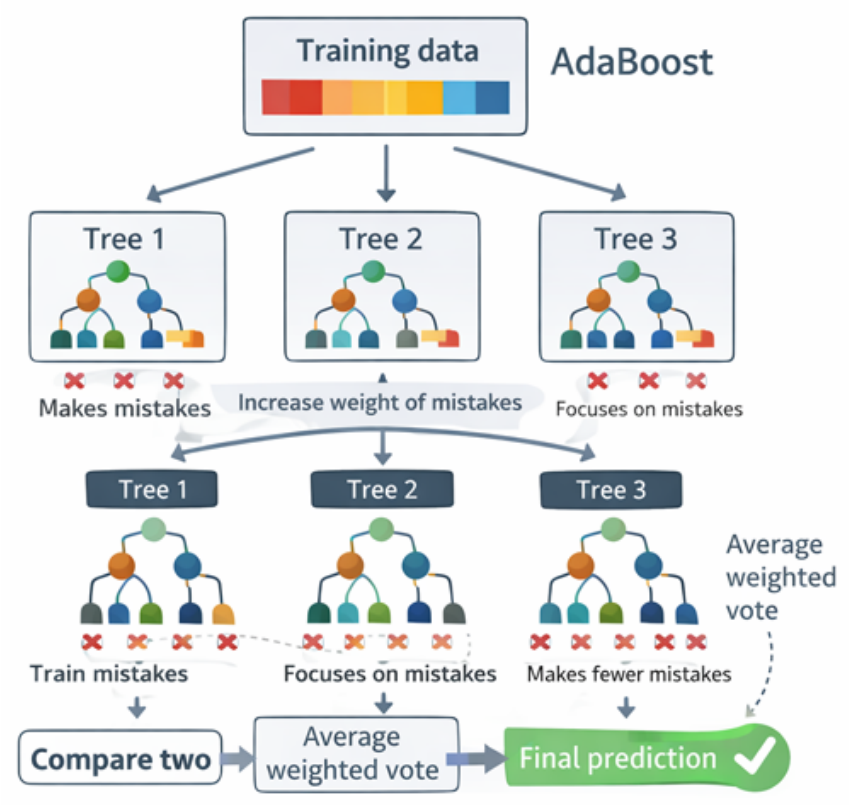
```
# Assumes these already exist from earlier code blocks:
# rf_acc, rf_ll (computed in your metrics block)
# results_df (created earlier and already contains prior model rows)
new_row = pd.DataFrame([{
  &quot;model&quot;: &quot;Random Forest&quot;,
  &quot;accuracy&quot;: rf_acc,
  &quot;log_loss&quot;: rf_ll
}])

# If the model already exists in the table, replace it; otherwise append it.
if (results_df[&quot;model&quot;] == &quot;Random Forest&quot;).any():
  results_df.loc[results_df[&quot;model&quot;] == &quot;Random Forest&quot;,
[&quot;accuracy&quot;, &quot;log_loss&quot;]] = [rf_acc, rf_ll]
else:
  results_df = pd.concat([results_df, new_row], ignore_index=True)
```

```
# Re-sort using your same rule
results_df = results_df.sort_values(by=[&quot;log_loss&quot;, &quot;accuracy&quot;], ascending=
[True, False])
results_df
```

	model	accuracy	log_loss
0	Logistic regression (OvR)	0.954198	0.129619
5	Random Forest	0.921279	0.194437
1	Bagging (100 trees, depth=3)	0.935592	0.194865
2	Decision tree (depth=3)	0.934160	0.215635
3	k-NN (k=15)	0.916031	0.569203
4	Naive Bayes (GaussianNB)	0.223760	27.978542

As you can see, logistic regression is the best model overall, followed by random forest and then bagging based on log loss.

14.5Boosting with AdaBoost



Bagging and random forests reduce error by training many models independently and averaging their predictions. Boosting takes a different approach by training models *sequentially*, where each new model focuses on correcting the mistakes made by the previous ones.

The most widely used classical boosting algorithm is **AdaBoost** (Adaptive Boosting). Instead of treating all observations equally, AdaBoost increases the importance of misclassified examples so that later models concentrate on the hardest cases.

Core ideas behind boosting

- **Sequential learning:** models are trained one after another, not in parallel.
- **Reweighting mistakes:** misclassified observations receive higher weights.
- **Weak learners:** each model is intentionally simple, often a shallow decision tree.

By combining many weak learners that are each only slightly better than random guessing, boosting constructs a strong classifier that can reduce both bias and variance.

AdaBoost algorithm overview

Table 14.5
AdaBoost classification procedure

Step	Description
1	Assign equal weights to all training observations.
2	Train a weak learner (usually a shallow decision tree) on the weighted data.
3	Increase the weights of misclassified observations and decrease the weights of correctly classified ones.
4	Repeat steps 2–3 for many rounds.
5	Combine all learners using a weighted vote based on their accuracy.

AdaBoost in scikit-learn

Scikit-learn implements AdaBoost using the *AdaBoostClassifier* class. We again reuse the same cleaned dataset, preprocessing pipeline, and train/test split so that performance can be compared directly with bagging and random forests.

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

base_tree = DecisionTreeClassifier(
  max_depth=1,
  random_state=27
)

model_adaboost = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;ada&quot;, AdaBoostClassifier(
      estimator=base_tree,
      n_estimators=200,
      learning_rate=0.5,
      random_state=27
  ))
])

model_adaboost.fit(X_train, y_train)
```

The base learner is intentionally restricted to a decision stump (*max_depth = 1*) so that each individual model is weak. The hyperparameter *n_estimators* controls how many boosting rounds are performed, while *learning_rate* determines how strongly each new learner influences the final model.

Smaller learning rates slow down learning but often improve generalization, while larger values can lead to faster convergence but higher risk of overfitting.

Comparing boosting to bagging and random forests

Bagging and random forests reduce error primarily by averaging many independent models to stabilize predictions. AdaBoost instead reduces error by repeatedly focusing on the hardest-to-classify observations.

In practice, boosting often achieves higher accuracy than bagging when patterns are subtle and nonlinear, but it can be more sensitive to noisy labels because mislabeled observations receive increasing weight over time.

The next ensemble method extends this idea further by replacing reweighting with direct optimization of a loss function using gradient descent, leading to gradient boosting models.

Deliverable: Add AdaBoost to the running model comparison table

To compare boosting fairly, we evaluate AdaBoost using the same train/test split, the same preprocessing pipeline, and the same evaluation function used for earlier models. We then append AdaBoost results to the running comparison table so we can see whether boosting improved accuracy, probability quality (log loss), or both.

```
def eval_model_row(name, model, X_train, y_train, X_test, y_test):
  model.fit(X_train, y_train)
  y_pred = model.predict(X_test)
  acc = accuracy_score(y_test, y_pred)
  ll = None

  if hasattr(model, &quot;predict_proba&quot;):
    y_prob = model.predict_proba(X_test)
    ll = log_loss(y_test, y_prob)

  return {&quot;model&quot;: name, &quot;accuracy&quot;: acc, &quot;log_loss&quot;: ll}

row = eval_model_row(
  &quot;AdaBoost (stumps, n=200)&quot;,
  model_adaboost,
  X_train, y_train,
  X_test, y_test
)
```

```
      # Upsert into the existing results_df (replace if present, otherwise append)
      if (results_df[&quot;model&quot;] == row[&quot;model&quot;]).any():
        results_df.loc[results_df[&quot;model&quot;] == row[&quot;model&quot;], [&quot;accuracy&quot;,
&quot;log_loss&quot;]] = [row[&quot;accuracy&quot;], row[&quot;log_loss&quot;]]
      else:
        results_df = pd.concat([results_df, pd.DataFrame([row])], ignore_index=True)

      # Re-sort using the same rule used throughout the chapter
      results_df = results_df.sort_values(by=[&quot;log_loss&quot;, &quot;accuracy&quot;], ascending=
[True, False])
      results_df
```

	model	accuracy	log_loss
0	Logistic regression (OvR)	0.954198	0.129619
1	Random Forest	0.921279	0.194437
2	Bagging (100 trees, depth=3)	0.935592	0.194865
3	Decision tree (depth=3)	0.934160	0.215635
6	AdaBoost (stumps, n=200)	0.950859	0.497037
4	k-NN (k=15)	0.916031	0.569203
5	Naive Bayes (GaussianNB)	0.223760	27.978542

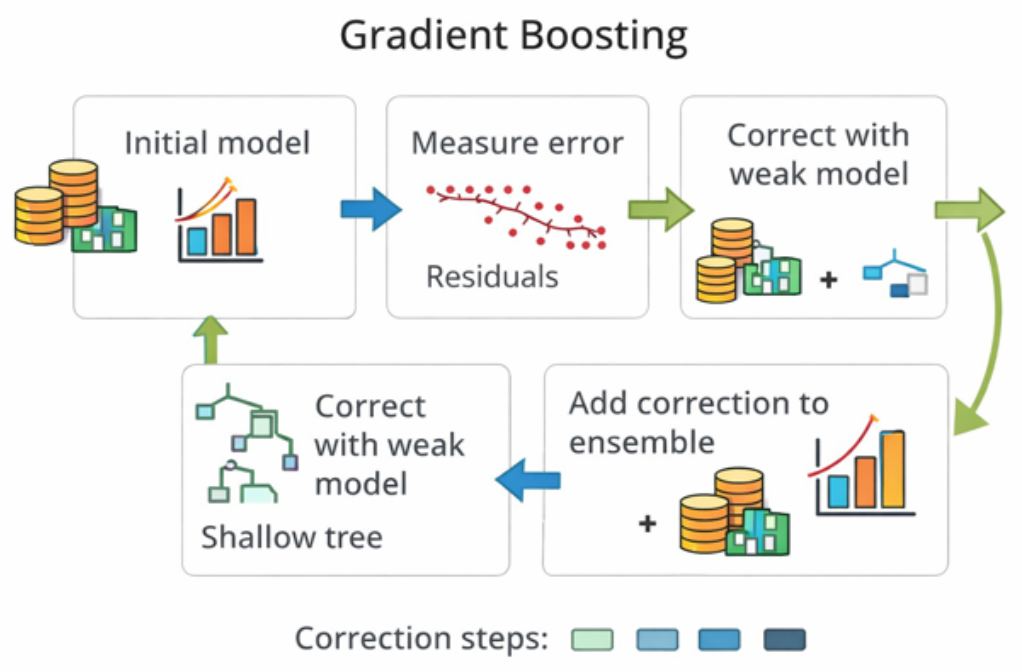
In this experiment, AdaBoost achieves an accuracy of *0.9509*, which is slightly higher than the single decision tree (*0.9342*), bagging (*0.9356*), and the random forest (*0.9213*). This reflects boosting's ability to reduce classification error by concentrating successive models on the observations that earlier models misclassified.

However, AdaBoost's log loss is *0.4970*, which is substantially worse than bagging (*0.1949*), random forests (*0.1944*), and logistic regression (*0.1296*). This indicates that although AdaBoost predicts the correct class more often, its probability estimates are less well calibrated and tend to be more overconfident on difficult cases. The mechanism behind this overconfidence is AdaBoost's reweighting strategy: as the algorithm repeatedly increases weights on misclassified observations, later trees become increasingly focused on correcting these hard cases. This can cause the

ensemble to assign extreme probabilities (very close to 0 or 1) to difficult observations, which hurts log loss even when the final class prediction is correct.

This contrast highlights a key tradeoff: boosting can deliver strong accuracy gains, but not necessarily better probability quality. The next ensemble method addresses this limitation by directly optimizing a loss function using gradient descent, which often improves both accuracy and probability calibration.

14.6Gradient Boosting



Boosting methods build strong predictors by combining many weak models in sequence. Instead of training many models independently and averaging them, gradient boosting trains each new model to correct the errors made by the ensemble so far.

For classification, the “errors” are defined by a loss function such as log loss, not just whether a case was misclassified. This makes gradient boosting especially valuable when you care about probability quality, not only the final predicted label.

How gradient boosting works

Gradient boosting can be understood as a repeated cycle of “predict, measure loss, and correct.” Each new tree is trained on the residual signal that remains after the current ensemble has already done its best.

- Start with a simple baseline prediction for every case. For classification, this often begins with the overall class rate (a constant probability).
- Compute how wrong the current model is using a loss function such as log loss. These loss gradients indicate which cases the model is currently under- or over-confident about.
- Train a shallow decision tree to predict the direction and size of the correction needed. This tree is intentionally weak so it improves the ensemble gradually.
- Scale the tree’s contribution using a learning rate. Smaller learning rates usually improve stability but require more trees.
- Add the scaled correction into the ensemble and repeat many times. Over iterations, many small fixes can produce a highly accurate model.

Compared to AdaBoost, gradient boosting is more general because it is framed as direct optimization of a loss function. That framing makes it easier to add regularization, tune probability quality, and extend the algorithm in production-grade systems.

Gradient boosting in scikit-learn

The code below trains a *GradientBoostingClassifier* using the same training data, preprocessing pipeline, and evaluation objects already created earlier. This keeps the comparison fair because differences in performance are driven by the algorithm rather than by changes in features, splits, or data preparation.

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score, log_loss, classification_report

model_gbdt = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;gbdt&quot;, GradientBoostingClassifier(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=3,
    subsample=1.0,
    random_state=27
```

```
    ))
])

model_gbdt.fit(X_train, y_train)
y_gbdt_pred = model_gbdt.predict(X_test)
y_gbdt_prob = model_gbdt.predict_proba(X_test)
gbdt_acc = accuracy_score(y_test, y_gbdt_pred)
gbdt_ll = log_loss(y_test, y_gbdt_prob)

print(&quot;GBDT accuracy:&quot;, round(gbdt_acc, 4))
print(&quot;GBDT log loss:&quot;, round(gbdt_ll, 4))
print(&quot;\nClassification report:&quot;)
print(classification_report(y_test, y_gbdt_pred, digits=3, zero_division=0))


# Output:
# GBDT accuracy: 0.968
# GBDT log loss: 0.0942
# Classification report:
#               precision    recall  f1-score   support
#           0      0.991     0.633     0.773       180
#           1      0.967     0.999     0.983      1916
#     accuracy                          0.968      2096
#   macro avg      0.979     0.816     0.878      2096
# weighted avg      0.969     0.968     0.965      2096
```

Key hyperparameters for GradientBoostingClassifier

Gradient boosting performance is driven by a small set of hyperparameters that control how many trees you add and how aggressively each one updates the ensemble. The goal is to add enough trees to learn meaningful patterns, but not so aggressively that the model becomes overconfident or overfits noise.

Table 14.6
Common GradientBoostingClassifier hyperparameters

Hyperparameter	What it controls	Typical effect
n_estimators	Number of boosting stages (trees) added sequentially.	More trees usually improves fit, but can overfit unless learning_rate is small.
learning_rate	How strongly each new tree updates the ensemble.	Lower values improve stability and calibration, but require more trees.
max_depth	Depth of each weak tree (interaction complexity per stage).	Shallow trees generalize better; deeper trees can overfit and become overconfident.
subsample	Fraction of training rows used per tree (stochastic gradient boosting).	Values below 1.0 can reduce overfitting and improve generalization.

Hyperparameter	What it controls	Typical effect
random_state	Controls reproducibility when stochastic elements exist (such as subsampling).	Keeping the seed fixed makes results repeatable for learners and grading.

In this chapter workflow, you can think of *learning_rate* and *n_estimators* as a pair. A smaller learning rate generally calls for a larger number of trees, which can improve probability quality when tuned carefully.

So far, we have used scikit-learn's *GradientBoostingClassifier*, which provides a clear and educational implementation of gradient boosting. However, in practice, many data scientists use specialized libraries such as XGBoost, LightGBM, or CatBoost. These are not different algorithms—they are optimized implementations of gradient boosting that add advanced regularization techniques, better computational efficiency, and more sophisticated hyperparameter controls. XGBoost (eXtreme Gradient Boosting) is perhaps the most widely used of these libraries and represents the same core gradient boosting idea we just learned, but with engineering improvements that often lead to better performance and faster training.

XGBoost vs. scikit-learn GradientBoostingClassifier

Scikit-learn's gradient boosting is an excellent educational implementation, but modern production systems often use optimized libraries such as XGBoost. XGBoost is still gradient boosting, but it adds engineering and regularization improvements that often lead to better accuracy, better probability control, and much faster training on large datasets.

Table 14.7
Conceptual differences: scikit-learn GBDT vs XGBoost

Aspect	scikit-learn GradientBoostingClassifier	XGBoost XGBClassifier
Regularization	Limited regularization controls compared to modern variants.	Built-in L2/L1-style regularization and split penalties to reduce overfitting.

Aspect	scikit-learn GradientBoostingClassifier	XGBoost XGBClassifier
Tree growth and splits	Traditional boosting approach with fewer tuning knobs.	Highly optimized split finding with additional parameters for controlling complexity.
Speed and scalability	Good for moderate datasets and learning workflows.	Highly optimized and often faster, especially with many features and many rows.
Handling sparsity	Works with sparse features via preprocessing, but not optimized for sparsity.	Designed to exploit sparsity efficiently, which is common after one-hot encoding.
Probability behavior	Often strong, but can require careful tuning for calibration.	Often strong and tunable, but can still become overconfident without regularization.

XGBoost in Python

The next code cell trains an XGBoost classifier using the same training and test data objects. The configuration below is intentionally conservative so it trains quickly and produces stable probabilities for comparison, and you can tune it later if needed.

```
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, log_loss, classification_report

model_xgb = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;xgb&quot;, XGBClassifier(
    objective=&quot;binary:logistic&quot;,
    eval_metric=&quot;logloss&quot;,
    n_estimators=400,
    learning_rate=0.05,
    max_depth=3,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_lambda=1.0,
    min_child_weight=1.0,
    gamma=0.0,
    n_jobs=-1,
    random_state=27
  ))
])

model_xgb.fit(X_train, y_train)
y_xgb_pred = model_xgb.predict(X_test)
y_xgb_prob = model_xgb.predict_proba(X_test)
xgb_acc = accuracy_score(y_test, y_xgb_pred)
xgb_ll = log_loss(y_test, y_xgb_prob)
```

```
print(&quot;XGBoost accuracy:&quot;, round(xgb_acc, 4))
print(&quot;XGBoost log loss:&quot;, round(xgb_ll, 4))
print(&quot;\nClassification report:&quot;)
print(classification_report(y_test, y_xgb_pred, digits=3, zero_division=0))


# Output:
# XGBoost accuracy: 0.9828
# XGBoost log loss: 0.0539
# Classification report:
#               precision    recall  f1-score   support
#           0      0.993     0.806     0.890       180
#           1      0.982     0.999     0.991      1916
#     accuracy                          0.983      2096
#   macro avg      0.988     0.903     0.940      2096
# weighted avg      0.983     0.983     0.982      2096
```

Key hyperparameters for XGBClassifier

XGBoost exposes additional hyperparameters that control tree complexity and regularization more explicitly. These settings help you balance accuracy with generalization and also help prevent overly confident probability predictions.

Table 14.8
Common XGBClassifier hyperparameters

Hyperparameter	What it controls	Typical effect
n_estimators	Number of boosting rounds (trees).	More trees increases fit; pair with a smaller learning_rate for stability.
learning_rate	Step size for each boosting update.	Lower values often improve calibration and reduce overfitting but require more trees.
max_depth	Maximum depth of each tree.	Controls interaction complexity; deeper trees can overfit.
subsample	Fraction of rows sampled for each tree.	Stochasticity reduces overfitting and can improve generalization.
colsample_bytree	Fraction of features sampled for each tree.	Feature randomness reduces correlation and overfitting, especially with many one-hot columns.
reg_lambda	L2-style regularization on leaf weights.	Higher values shrink weights and reduce overconfidence and variance.
min_child_weight	Minimum “mass” required in a child node for a split.	Higher values make splitting harder and can reduce overfitting.

Hyperparameter	What it controls	Typical effect
gamma	Minimum loss reduction required to make a split.	Higher values discourage small, noisy splits and improve generalization.
random_state	Controls reproducibility when sampling is used.	Fixed seeds help ensure consistent results across runs and learners.

Adding both models to the cumulative comparison table

To keep your model comparison table cumulative, do not rebuild it from scratch. Instead, append new rows into the existing *results_df*, then re-sort by log loss and accuracy so the best probability models rise to the top.

```
import numpy as np

def append_result_row(results_df, model_name, acc, ll):
  results_df.loc[len(results_df)] = {
    &quot;model&quot;: model_name,
    &quot;accuracy&quot;: float(acc),
    &quot;log_loss&quot;: float(ll) if ll is not None and not np.isnan(ll) else np.nan
  }

  results_df = results_df.sort_values(
    by=[&quot;log_loss&quot;, &quot;accuracy&quot;],
    ascending=[True, False]
  ).reset_index(drop=True)

  return results_df

results_df = append_result_row(results_df, &quot;Gradient Boosting (GBDT)&quot;, gbdt_acc,
gbdt_ll)
results_df = append_result_row(results_df, &quot;XGBoost (XGBClassifier)&quot;, xgb_acc, xgb_ll)
results_df
```

	model	accuracy	log_loss
0	XGBoost (XGBClassifier)	0.982824	0.053891
1	Gradient Boosting (GBDT)	0.968034	0.094194
2	Logistic regression (OvR)	0.954198	0.129619
3	Random Forest	0.921279	0.194437
4	Bagging (100 trees, depth=3)	0.935592	0.194865
5	Decision tree (depth=3)	0.934160	0.215635
6	AdaBoost (stumps, n=200)	0.950859	0.497037
7	k-NN (k=15)	0.916031	0.569203
8	Naive Bayes (GaussianNB)	0.223760	27.978542

Interpreting the Gradient Boosting Results

The updated comparison table shows that gradient boosting methods produced the strongest overall performance on this Lending Club prediction task. The best model by both accuracy and log loss is **XGBoost (XGBClassifier)**, with *accuracy = 0.9828* and *log loss = 0.0539*, indicating both excellent classification performance and very high-quality probability estimates.

The scikit-learn **Gradient Boosting (GBDT)** model also performs extremely well with *accuracy = 0.9680* and *log loss = 0.0942*. This places it ahead of every earlier model in the chapter, including the best single-model baseline, *logistic regression* (*accuracy = 0.9542*, *log loss = 0.1296*), which had previously been the top performer for probability quality.

A key takeaway is that gradient boosting improves both goals simultaneously: it increases accuracy and also lowers log loss. For example, compared to logistic

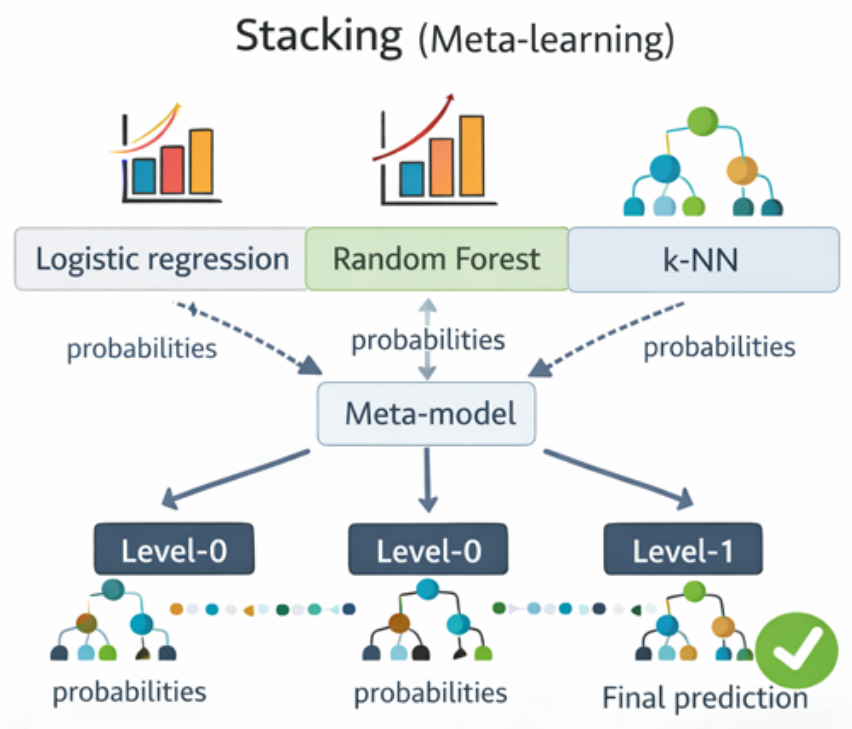
regression, GBDT improves accuracy by about *0.0138* (0.9680 − 0.9542) while reducing log loss by about *0.0354* (0.1296 − 0.0942), and XGBoost improves even further with an additional log loss reduction of about *0.0403* relative to GBDT (0.0942 − 0.0539).

In contrast, bagging-style methods improve probability quality modestly but do not match boosting on this dataset. *Random Forest* (*accuracy = 0.9213, log loss = 0.1944*) and *Bagging* (*accuracy = 0.9356, log loss = 0.1949*) reduce variance through averaging, but their probability estimates remain less precise than the boosted models, which are explicitly optimizing log loss during training.

AdaBoost's results illustrate an important caution about probability quality. Even though *AdaBoost (stumps, n=200)* achieves high accuracy (*0.9509*), its log loss is much worse (*0.4970*), which suggests overconfident or poorly calibrated probabilities for at least some cases.

Overall, the table supports a practical conclusion: when your business decision depends on reliable probability scores (such as pricing risk, prioritizing reviews, or setting thresholds), gradient boosting is often a top choice. Among gradient boosting implementations, XGBoost frequently leads because it adds regularization and optimization features that improve generalization and probability behavior, which is consistent with its strong showing here.

14.7Stacking



Bagging and boosting combine models by averaging or sequentially correcting errors. **Stacking** (also called **meta-learning**) combines models by training a second model to learn how to best mix their predictions.

The idea is to treat each base model as a feature generator. Each base model makes a prediction (often a probability), and a **meta-model** learns how to weight those predictions to make a final decision.

Why stacking can outperform single models

Different algorithms tend to make different kinds of mistakes. A stacking ensemble can learn patterns like “trust the linear model when the signal is mostly additive” and “trust the tree-based model when interactions matter.”

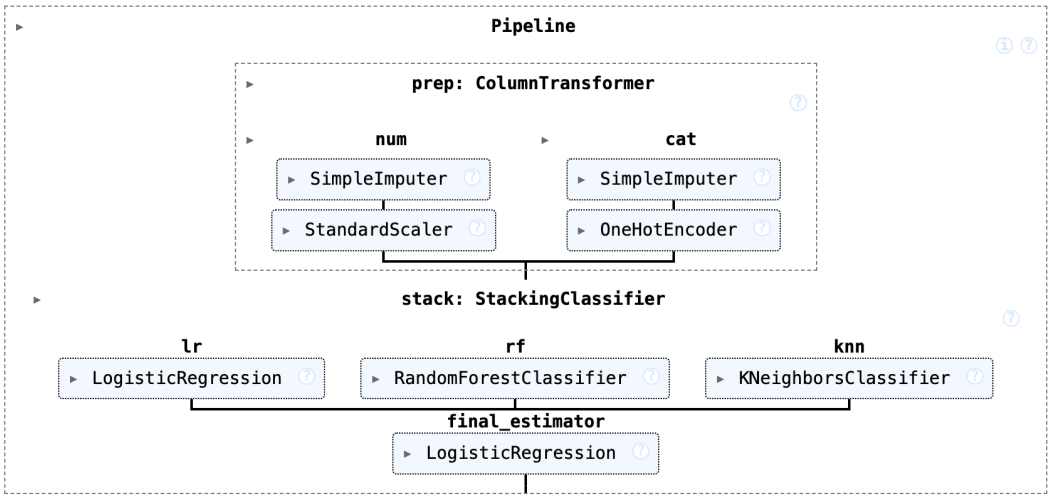
The biggest risk is **information leakage**. If the meta-model is trained on base-model predictions made from the same training rows that fit those base models, the meta-model can overfit by learning overly optimistic predictions.

To prevent leakage, stacking uses **out-of-fold predictions**: the base models generate predictions for each training row only when that row was held out from the base model's training fold. Scikit-learn handles this automatically.

Stacking in scikit-learn

In this example, we reuse the same train/test split and the same *preprocessor* pipeline. We will combine three base learners (logistic regression, random forest, and k-NN) and then train a logistic regression meta-model on their predicted probabilities.

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline

base_lr = LogisticRegression(
  solver=&quot;liblinear&quot;,
  max_iter=2000,
  random_state=27
)

base_rf = RandomForestClassifier(
  n_estimators=200,
  max_depth=None,
  min_samples_leaf=1,
  n_jobs=-1,
  random_state=27
)

base_knn = KNeighborsClassifier(n_neighbors=15)

meta_lr = LogisticRegression(
  solver=&quot;liblinear&quot;,
  max_iter=2000,
  random_state=27
)

model_stacking = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;stack&quot;, StackingClassifier(
      estimators=[
        (&quot;lr&quot;, base_lr),
        (&quot;rf&quot;, base_rf),
        (&quot;knn&quot;, base_knn)
      ],
      final_estimator=meta_lr,
      stack_method=&quot;predict_proba&quot;,
      cv=5,
      n_jobs=-1,
      passthrough=False
  ))
])

model_stacking.fit(X_train, y_train)
```



Interpreting the key hyperparameters

The *estimators* argument defines the level-0 learners. Each is trained using cross-validation folds so that the meta-model sees out-of-fold predictions rather than in-sample predictions.

The *final_estimator* is the meta-model. We use logistic regression because it is fast, stable, and produces well-behaved probability estimates. Conceptually, it learns how to weight base-model probabilities to improve decisions.

The *stack_method="predict_proba"* option tells stacking to feed predicted probabilities (not class labels) into the meta-model. This is usually preferred when you care about log loss and probability quality.

The *cv* argument controls how out-of-fold predictions are generated. More folds can reduce leakage risk and stabilize the meta-features but increases training time because base learners are refit multiple times.

Setting *passthrough=False* means the meta-model uses only base-model outputs. If you set it to True, the original features are also passed to the meta-model, which can improve performance but increases overfitting risk and reduces interpretability.

Adding stacking results to the chapter comparison table

Next, we evaluate the stacking model on the test set and add its accuracy and log loss as a new row in the existing *results_df* so we can compare it to every prior model using the same evaluation setup.

```
from sklearn.metrics import accuracy_score, log_loss

y_stack_pred = model_stacking.predict(X_test)
stack_acc = accuracy_score(y_test, y_stack_pred)
stack_ll = None

if hasattr(model_stacking, &quot;predict_proba&quot;):
  y_stack_prob = model_stacking.predict_proba(X_test)
  stack_ll = log_loss(y_test, y_stack_prob)

row = {&quot;model&quot;: &quot;Stacking (LR + RF + kNN → LR)&quot;, &quot;accuracy&quot;:
stack_acc, &quot;log_loss&quot;: stack_ll}

if &quot;results_df&quot; not in globals():
  results_df = pd.DataFrame([row])
else:
  results_df = pd.concat([results_df, pd.DataFrame([row])], ignore_index=True)

results_df = results_df.sort_values(by=[&quot;log_loss&quot;, &quot;accuracy&quot;], ascending=
[True, False]).reset_index(drop=True)
results_df
```

	model	accuracy	log_loss
0	XGBoost (XGBClassifier)	0.982824	0.053891
1	Gradient Boosting (GBDT)	0.968034	0.094194
2	Logistic regression (OvR)	0.954198	0.129619
3	Stacking (LR + RF + kNN → LR)	0.956107	0.136311
4	Random Forest	0.921279	0.194437
5	Bagging (100 trees, depth=3)	0.935592	0.194865
6	Decision tree (depth=3)	0.934160	0.215635
7	AdaBoost (stumps, n=200)	0.950859	0.497037
8	k-NN (k=15)	0.916031	0.569203
9	Naive Bayes (GaussianNB)	0.223760	27.978542

In this run, the stacking ensemble (*Stacking (LR + RF + kNN → LR)*) achieved a test-set accuracy of *0.9561* with a log loss of *0.1363*. This places stacking slightly above the standalone logistic regression model in accuracy (*0.9542*) but with a slightly worse log loss than logistic regression (*0.1296*), meaning stacking made marginally fewer classification mistakes at the 0.50 threshold but produced slightly less well-calibrated probabilities overall.

Compared to variance-reduction methods, stacking is clearly competitive. It outperforms random forests (*0.9213* accuracy, *0.1944* log loss) and bagging (*0.9356* accuracy, *0.1949* log loss) on both metrics, suggesting that combining diverse base learners can improve both classification performance and probability quality. It also substantially improves on a single shallow tree (*0.9342* accuracy, *0.2156* log loss), reinforcing the idea that ensembles can stabilize weak or high-variance learners without requiring deeper trees.

However, the strongest probability models in this comparison are still the gradient boosting family. The gradient boosting tree model (*0.9680* accuracy, *0.0942* log loss) and especially XGBoost (*0.9828* accuracy, *0.0539* log loss) outperform stacking by a meaningful margin. This indicates that, for this dataset, sequential boosting is capturing important nonlinear structure and producing better-calibrated risk estimates than a meta-learner combining a small set of heterogeneous models.

Operationally, the stacking result is still valuable because it demonstrates a practical middle ground: it provides strong accuracy and good log loss without requiring a specialized boosting library, and it offers a simple “mixture of experts” intuition (the meta-model learns which base learner to trust in different regions of the feature space). But if the business objective prioritizes probability quality for ranking, pricing, or threshold tuning, the lower log loss values from gradient boosting methods (especially *0.0539* for XGBoost) provide a strong argument for choosing boosting over stacking in this particular setting.

When is stacking most beneficial? Stacking tends to provide the greatest value in the following scenarios:

- When base learners make complementary errors—that is, when different models make mistakes on different observations, allowing the meta-learner to combine their strengths.
- When you have diverse model families (linear + tree-based + distance-based) that capture different aspects of the data structure, rather than multiple similar models.
- When probability quality matters more than training speed, since stacking requires refitting base learners multiple times during cross-validation, which can be computationally expensive.

Implementation tips
If stacking takes too long, reduce *cv* (for example, from 5 to 3) or simplify base learners (fewer random forest trees, smaller k for k-NN). Stacking refits base learners multiple times, so training time can increase quickly.

Conversely, when you have more data and want better out-of-fold predictions for the meta-learner, consider increasing *cv* (for example, from 5 to 10). More folds provide more training data for the meta-learner and can improve its ability to learn how to combine base learners effectively, though at the cost of longer training time.

14.8Ensembles for Regression

All of the ensemble concepts we have covered in this chapter—bagging, random forests, boosting, and stacking—apply equally well to regression problems. The core principles remain the same: combining multiple models reduces variance, improves stability, and often leads to better predictions. The main difference is that regression ensembles average numeric predictions (such as predicted prices, sales volumes, or continuous outcomes) rather than combining class probabilities. This means that instead of voting on class labels or averaging probabilities, regression ensembles typically compute the mean or weighted mean of the numeric predictions from each base model.

In classification, ensembles usually combine predicted probabilities and then choose the most likely class. In regression, ensembles combine predicted numeric values, often by averaging (bagging/random forests) or by adding incremental corrections (boosting).

Table 14.9

Ensemble methods for classification and regression

Ensemble idea	Classification version	Regression version	Best-fit scenarios
Bagging	BaggingClassifier	BaggingRegressor	High-variance base learners (especially trees); you want stability and robust performance.
Random forest	RandomForestClassifier	RandomForestRegressor	Strong general-purpose baseline for tabular data; nonlinear relationships; feature importance needed.
AdaBoost	AdaBoostClassifier	AdaBoostRegressor	When simple learners can be combined to capture structure; can be sensitive to outliers/noise.
Gradient boosting	GradientBoostingClassifier	GradientBoostingRegressor	Often excellent accuracy on tabular data; good when you can tune learning rate and tree count.
XGBoost (industry GBDT)	XGBClassifier	XGBRegressor	When you want top-tier performance and advanced regularization; common in competitions and production.

Ensemble idea	Classification version	Regression version	Best-fit scenarios
Stacking	StackingClassifier	StackingRegressor	When different model families make complementary errors; requires careful cross-validation to avoid leakage.

To illustrate regression ensembles in a realistic business case, we will reuse the **insurance** dataset from earlier regression work. The goal is to predict *charges* (annual medical insurance cost) from customer attributes such as age, BMI, smoking status, and region.

For regression, our evaluation metrics change. We will use *RMSE* to measure typical prediction error in dollars, *MAE* to measure absolute error robustness, and R^2 to measure variance explained.

Step 1: Load data, split train/test, and build a reusable preprocessing pipeline

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer

df = pd.read_csv(&quot;/content/drive/MyDrive/Colab Notebooks/data/insurance.csv&quot;)
y = df[&quot;charges&quot;].copy()
X = df.drop(columns=[&quot;charges&quot;]).copy()

X_train, X_test, y_train, y_test = train_test_split(
  X, y,
  test_size=0.20,
  random_state=27
)

cat_cols = X_train.select_dtypes(include=[&quot;object&quot;, &quot;category&quot;,
&quot;bool&quot;]).columns.tolist()
num_cols = X_train.select_dtypes(include=[&quot;number&quot;]).columns.tolist()

num_pipe = Pipeline(steps=[
  (&quot;imp&quot;, SimpleImputer(strategy=&quot;median&quot;)),
```

```
    (&quot;scaler&quot;, StandardScaler())
  ])

  cat_pipe = Pipeline(steps=[
    (&quot;imp&quot;, SimpleImputer(strategy=&quot;most_frequent&quot;)),
    (&quot;ohe&quot;, OneHotEncoder(handle_unknown=&quot;ignore&quot;))
  ])

  preprocessor = ColumnTransformer(
    transformers=[
      (&quot;num&quot;, num_pipe, num_cols),
      (&quot;cat&quot;, cat_pipe, cat_cols)
    ],
    remainder=&quot;drop&quot;
  )
```

The preprocessing pipeline handles missing data through imputation and converts categorical attributes into one-hot encoded features. Scaling is applied to numeric features to support algorithms like k-NN and linear regression, while tree-based ensembles will typically be insensitive to the scaling step.

Step 2: Define a reusable evaluation function and a results table

```
  import time
  from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

  results_reg_df = pd.DataFrame(columns=[&quot;model&quot;, &quot;rmse&quot;, &quot;mae&quot;,
&quot;r2&quot;, &quot;train_seconds&quot;])

  def upsert_result_row_reg(results_df, row):
    if (results_df[&quot;model&quot;] == row[&quot;model&quot;]).any():
      idx = results_df.index[results_df[&quot;model&quot;] == row[&quot;model&quot;]][0]

      for k, v in row.items():
        results_df.at[idx, k] = v
    else:
      results_df = pd.concat([results_df, pd.DataFrame([row])], ignore_index=True)

    results_df = results_df.sort_values(
      by=[&quot;rmse&quot;, &quot;r2&quot;],
      ascending=[True, False]
    ).reset_index(drop=True)

    return results_df

  def eval_and_update_results_df_reg(model_name, model, X_train, y_train, X_test, y_test,
results_df):
    t0 = time.perf_counter()
    model.fit(X_train, y_train)
    train_seconds = time.perf_counter() - t0
    y_pred = model.predict(X_test)
    rmse = float(np.sqrt(mean_squared_error(y_test, y_pred)))
    mae = float(mean_absolute_error(y_test, y_pred))
    r2 = float(r2_score(y_test, y_pred))

    row = {
      &quot;model&quot;: model_name,
      &quot;rmse&quot;: rmse,
      &quot;mae&quot;: mae,
      &quot;r2&quot;: r2,
```

```
    &quot;train_seconds&quot;: train_seconds
  }

  results_df = upsert_result_row_reg(results_df, row)

  return results_df
```

RMSE and MAE are measured in the same units as the label, which makes them easy to interpret in business terms. R^2 is dimensionless and summarizes the share of variation in charges explained by the model.

Step 3: Train baseline regressors and ensemble regressors

The models below mirror the families we used for classification, but in regression form. After running the cell, you will have a single comparison table that includes both single-model baselines and ensembles.

```
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.ensemble import BaggingRegressor, RandomForestRegressor, AdaBoostRegressor
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.ensemble import StackingRegressor

model_lr_reg = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;lr&quot;, LinearRegression())
])

model_tree_reg = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;tree&quot;, DecisionTreeRegressor(
    max_depth=3,
    random_state=27
  ))
])

model_knn_reg = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;knn&quot;, KNeighborsRegressor(n_neighbors=15))
])

base_tree_reg = DecisionTreeRegressor(
  max_depth=3,
  random_state=27
)

model_bag_reg = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;bag&quot;, BaggingRegressor(
    estimator=base_tree_reg,
    n_estimators=200,
    bootstrap=True,
    n_jobs=-1,
    random_state=27
  ))
])
```

```
model_rf_reg = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;rf&quot;, RandomForestRegressor(
    n_estimators=300,
    max_depth=None,
    min_samples_leaf=1,
    n_jobs=-1,
    random_state=27
  ))
])

model_ada_reg = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;ada&quot;, AdaBoostRegressor(
    estimator=DecisionTreeRegressor(max_depth=2, random_state=27),
    n_estimators=300,
    learning_rate=0.05,
    loss=&quot;linear&quot;,
    random_state=27
  ))
])

model_gbdt_reg = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;gbdt&quot;, GradientBoostingRegressor(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=3,
    random_state=27
  ))
])

stack_base_lr = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;lr&quot;, LinearRegression())
])

stack_base_rf = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;rf&quot;, RandomForestRegressor(
    n_estimators=200,
    n_jobs=-1,
    random_state=27
  ))
])

stack_base_knn = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;knn&quot;, KNeighborsRegressor(n_neighbors=25))
])

model_stack_reg = StackingRegressor(
  estimators=[
    (&quot;lr&quot;, stack_base_lr),
    (&quot;rf&quot;, stack_base_rf),
    (&quot;knn&quot;, stack_base_knn)
  ],
  final_estimator=LinearRegression(),
  cv=5,
  n_jobs=-1
)

try:
  from xgboost import XGBRegressor

  model_xgb_reg = Pipeline(steps=[
    (&quot;prep&quot;, preprocessor),
    (&quot;xgb&quot;, XGBRegressor(
      n_estimators=600,
      learning_rate=0.05,
      max_depth=4,
```

```
      subsample=0.9,
      colsample_bytree=0.9,
      reg_lambda=1.0,
      random_state=27,
      n_jobs=-1
    ))
  ])

except Exception as e:
  model_xgb_reg = None
  print(&quot;XGBoost not available in this environment:&quot;, e)
```

Stacking is handled slightly differently in regression because we combine predicted numeric values. The key leakage control is *cv=5*, which forces the meta-model to train on out-of-fold predictions rather than predictions produced on the same rows used to train a base learner.

Step 4: Add every model to the regression comparison table

```
models_reg_to_compare = [
  (&quot;Linear regression&quot;, model_lr_reg),
  (&quot;Decision tree (depth=3)&quot;, model_tree_reg),
  (&quot;k-NN (k=15)&quot;, model_knn_reg),
  (&quot;Bagging (200 trees, depth=3)&quot;, model_bag_reg),
  (&quot;Random forest&quot;, model_rf_reg),
  (&quot;AdaBoost (trees, n=300)&quot;, model_ada_reg),
  (&quot;Gradient boosting (GBDT)&quot;, model_gbdt_reg),
  (&quot;Stacking (LR + RF + kNN → LR)&quot;, model_stack_reg)
]

if model_xgb_reg is not None:
  models_reg_to_compare.append((&quot;XGBoost (XGBRegressor)&quot;, model_xgb_reg))

for name, model in models_reg_to_compare:
  results_reg_df = eval_and_update_results_df_reg(
    name, model,
    X_train, y_train,
    X_test, y_test,
    results_reg_df
  )

results_reg_df
```

	model	rmse	mae	r2	train_seconds
0	Gradient boosting (GBDT)	4994.078623	2666.504985	0.830562	0.765892
1	Bagging (200 trees, depth=3)	5182.865759	2985.282151	0.817509	0.470383
2	XGBoost (XGBRegressor)	5275.971267	2930.989663	0.810894	0.180526
3	Decision tree (depth=3)	5293.527747	3181.566247	0.809633	0.023974
4	Stacking (LR + RF + kNN → LR)	5304.556609	2777.586618	0.808839	5.687578
5	Random forest	5432.114875	2864.073686	0.799535	1.473218
6	AdaBoost (trees, n=300)	5774.776484	4610.071450	0.773446	0.552731
7	Linear regression	6282.054240	4271.668184	0.731895	0.020441
8	k-NN (k=15)	6693.257406	4074.359483	0.695648	0.022312

The results show that ensemble methods outperform single models for predicting insurance charges, with gradient boosting achieving the strongest overall performance.

Gradient boosting produced the lowest test RMSE (4,994) and MAE (2,667) and the highest R^2 value (0.831), indicating the most accurate and well-calibrated predictions among all models evaluated.

Bagging and XGBoost followed closely, with RMSE values of 5,183 and 5,276 respectively, demonstrating that averaging and boosting both substantially reduce error compared to a single decision tree.

The shallow decision tree performed surprisingly well given its simplicity, but it was still outperformed by all major ensemble approaches, particularly boosting-based models.

Stacking achieved similar accuracy to random forests but required significantly more training time, illustrating a common tradeoff between marginal accuracy gains and computational cost.

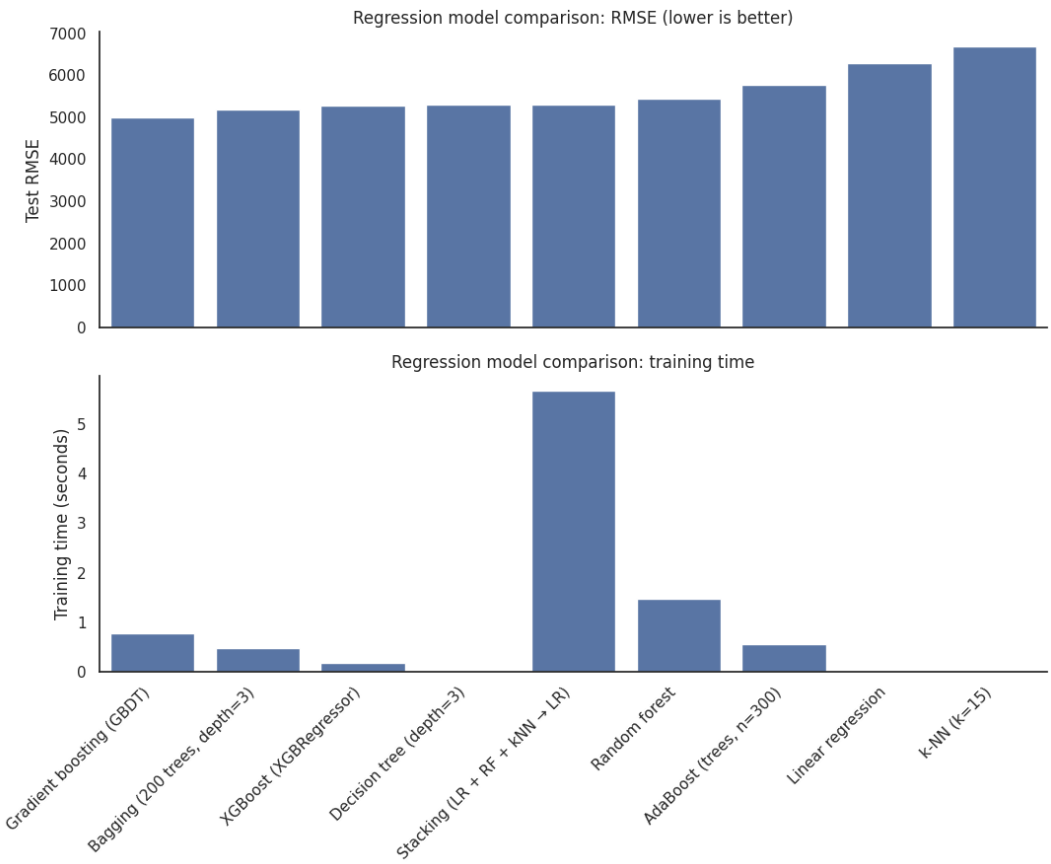
AdaBoost improved upon linear regression and k-NN but underperformed compared to newer boosting variants, suggesting sensitivity to noise and limited capacity when modeling complex cost drivers.

Overall, the results reinforce that boosting-based ensembles offer the best balance of predictive accuracy and practical efficiency for structured regression problems such as healthcare cost estimation.

Step 5: Visualize RMSE and training time

In real projects, model choice is not only about accuracy. Training time and operational complexity matter, especially when models must be retrained frequently or deployed in cost-sensitive environments.

```
import matplotlib.pyplot as plt
import seaborn as sns

plot_df = results_reg_df.copy()
sns.set_theme(style=&quot;white&quot;)
fig, axes = plt.subplots(nrows=2, ncols=1, figsize=(11, 9), sharex=True)

# RMSE plot
sns.barplot(
  data=plot_df,
  x=&quot;model&quot;,
  y=&quot;rmse&quot;,
  ax=axes[0]
)

axes[0].set_title(&quot;Regression model comparison: RMSE (lower is better)&quot;)
axes[0].set_ylabel(&quot;Test RMSE&quot;)

# Training time plot
sns.barplot(
  data=plot_df,
  x=&quot;model&quot;,
  y=&quot;train_seconds&quot;,
  ax=axes[1]
)

axes[1].set_title(&quot;Regression model comparison: training time&quot;)
axes[1].set_ylabel(&quot;Training time (seconds)&quot;)
axes[1].set_xlabel(&quot;&quot;)
sns.despine(top=True, right=True)
plt.xticks(rotation=45, ha=&quot;right&quot;)
plt.tight_layout()
plt.show()
```



The RMSE comparison shows a clear performance advantage for boosting-based ensemble methods on the insurance cost prediction task.

Gradient boosting achieves the lowest test error at approximately 4,994, followed by bagging at about 5,183 and XGBoost at about 5,276, while linear regression and k-NN perform worst with RMSE values above 6,200 and 6,600 respectively.

The training-time comparison reveals substantial computational differences, with stacking requiring nearly 5.7 seconds to train, random forests about 1.5 seconds, and gradient boosting and bagging under one second, while linear regression, k-NN, and a single decision tree train in only a few hundredths of a second.

These results illustrate a fundamental tradeoff between predictive accuracy and computational cost when selecting ensemble models.

If prediction accuracy is the primary objective, gradient boosting is the best choice because it delivers the lowest error with relatively modest training time.

If training speed and simplicity are critical, a shallow decision tree or linear regression model may be preferable despite their higher error.

Stacking provides strong performance but is difficult to justify operationally in this case because its error is higher than gradient boosting while its training time is several times longer.

Overall, gradient boosting represents the best balance of accuracy, stability, and computational efficiency for this regression problem, making it the recommended default ensemble approach for structured cost prediction tasks.

14.9Overfitting, Interpretability, and Tradeoffs

Ensemble models often deliver higher predictive accuracy than single models, but this improvement comes with important practical tradeoffs. In real systems, model quality must be balanced against interpretability, computational cost, memory usage, and prediction latency.

Understanding these tradeoffs is essential for choosing models that perform well not only in experiments, but also in production environments.

Ensembles vs single models

Single models such as linear regression or shallow decision trees are easy to train, fast to deploy, and straightforward to explain. However, they often suffer from high bias or high variance, limiting their predictive performance.

Ensembles reduce these limitations by combining many models, stabilizing predictions and capturing more complex patterns. This typically lowers error and improves probability calibration, but increases system complexity.

Explainability

Interpretability refers to how easily humans can understand why a model made a particular prediction. This is especially important in regulated domains such as lending, healthcare, and insurance.

- Linear models provide direct coefficient-based explanations.
- Decision trees offer rule-based explanations through their splits.
- Random forests and boosting models provide global feature importance but limited local explanations.
- Advanced tools such as *SHAP* and *LIME* can approximate local explanations for complex ensembles, but add additional computational overhead.

Computation, memory, and latency

Ensembles require more computation during both training and prediction because many models must be evaluated for each case. This can affect scalability in high-volume or real-time systems.

The table below provides a rough comparison of typical training times relative to a single decision tree, assuming similar hyperparameters and dataset size. These are approximate guidelines; actual times depend heavily on dataset size, feature count, tree depth, and hardware:

Table 14.10
Typical training time comparison (relative to a single decision tree)

Ensemble method	Typical training time (relative to single tree)
Single decision tree	1x (baseline)
Bagging (100 trees)	~100x (parallelizable, scales linearly with tree count)
Random Forest (100 trees)	~100x (parallelizable, similar to bagging)
AdaBoost (100 trees)	~100x (sequential, but trees are typically shallow)
Gradient Boosting (100 trees)	~100x (sequential, but can be parallelized within each tree)
XGBoost (100 trees)	~50-80x (optimized implementation, faster than scikit-learn GBDT)

Ensemble method	Typical training time (relative to single tree)
Stacking (3 base learners, cv=5)	~15-20x (must refit base learners multiple times for cross-validation)

Memory usage also increases because dozens or hundreds of trees must be stored, compared to a small set of coefficients for linear models.

Prediction latency becomes critical in applications such as fraud detection or recommendation systems, where decisions must be made in milliseconds. In these settings, simpler models may outperform ensembles operationally even if their statistical accuracy is lower.

Model selection decision guide

Table 14.11
Choosing between single models and ensembles

Criterion	Linear models	Decision trees	Bagging / Random Forest	Boosting / Stacking
Predictive accuracy	Low–moderate	Moderate	High	Very high
Overfitting risk	Low	High	Low	Low–moderate
Interpretability	Very high	High	Moderate	Low
Training cost	Very low	Low	Moderate	High
Prediction latency	Very low	Low	Moderate	High
Best use cases	Baseline models, high transparency	Rule extraction, simple logic	General-purpose production systems	High-stakes accuracy-critical applications

In practice, model selection is rarely about choosing the single most accurate algorithm.

Instead, practitioners must weigh performance gains against operational constraints, regulatory requirements, and user trust.

Ensembles often dominate in offline benchmarking, but simpler models may be preferred when transparency, speed, or maintainability are paramount.

A well-designed analytics system therefore treats ensemble methods as powerful tools rather than universal solutions.

14.10When NOT to use ensembles

While ensemble methods often improve predictive performance, there are several scenarios where they may not be worth the added complexity, cost, or tradeoffs:

- **Very small datasets (< 1000 rows):** With limited data, ensembles may overfit more easily than simpler models. The bootstrap sampling in bagging and random forests can create too much overlap in training sets, reducing diversity. A single well-regularized model (such as logistic regression with regularization or a shallow decision tree) may generalize better.
- **When interpretability is legally required:** In regulated industries such as finance (fair lending), healthcare (diagnostic explanations), and hiring (anti-discrimination laws), stakeholders must be able to explain model decisions. Ensemble methods, especially deep boosting or stacking, are "black boxes" that require additional explainability tools (such as SHAP or LIME) to meet regulatory requirements. A simple linear model or single decision tree may be preferable when direct interpretability is mandatory.
- **Real-time prediction systems with strict latency requirements:** Applications such as fraud detection, autonomous vehicles, or high-frequency trading require predictions in milliseconds. Ensembles must evaluate many models sequentially or in parallel, increasing latency. A single fast model (such as logistic regression or a shallow tree) may meet accuracy requirements while staying within latency budgets.
- **When a simple linear model already performs well:** If logistic regression or linear regression achieves acceptable accuracy (for example, 95%+ accuracy or low RMSE) and meets business requirements, the marginal improvement from ensembles may not justify the added complexity, training time, and maintenance burden. Always compare ensembles to simple baselines before assuming complexity is necessary.

The key principle is to start simple and add complexity only when it provides meaningful business value. Ensemble methods are powerful tools, but they are not always the right solution.

14.11Case Studies

See what you can learn from the practice problems below:

Case #1: Customer Churn (Ensemble Classification)
This case uses the same **Customer Churn** dataset from the previous chapter, but extends the analysis using **ensemble classification models**. Your goal is to examine how bagging, random forests, boosting, and stacking compare to single models for predicting customer churn.

Dataset attribution: Telecommunications customer churn dataset with demographics, service usage, contract attributes, and a binary churn outcome variable. [See details on Kaggle.com] The customer churn dataset is [available in the prior chapter] if you need to reload it.

Prediction goal: Predict whether a customer will churn (*Yes* or *No*) using all available features except the target variable.

For reproducibility, use **random_state = 27** everywhere a random seed is accepted.

Tasks

- Reuse the same preprocessing pipeline and train/test split from Chapter 13.
- Train a *RandomForestClassifier* and report accuracy, precision, recall, ROC AUC, and log loss.
- Train a *BaggingClassifier* with shallow decision trees as base learners and evaluate the same metrics.
- Train an *AdaBoostClassifier* using decision stumps.
- Train a *GradientBoostingClassifier*.
- Train a *StackingClassifier* using logistic regression, random forest, and k-NN as base learners.

- Add all models to a shared comparison table including accuracy, log loss, and macro-averaged F1 score.
- Plot confusion matrices for the two best-performing models.

Analytical questions

1. Which ensemble method achieved the highest accuracy?
2. Which model produced the lowest log loss?
3. Did any ensemble significantly improve recall for the churn class?
4. How did stacking compare to the best single model?
5. Which model would you deploy in a real telecom retention system and why?

Customer Churn Ensemble Models – Case Study Answers

These answers assume you used the Customer Churn dataset, an 80/20 stratified train/test split with **random_state = 27**, and identical preprocessing pipelines for all models. The values referenced below come directly from the evaluation results you reported for logistic regression, decision trees, bagging, random forests, AdaBoost, gradient boosting, and stacking.

Q1. Highest accuracy among ensemble methods

Among the ensemble models, the highest test-set accuracy was achieved by **stacking (LR + RF + kNN → LR)** with **accuracy = 0.7949**. This was slightly higher than gradient boosting (*0.7942*) and AdaBoost (*0.7928*), but still marginally below logistic regression's accuracy of *0.7970*.

Q2. Lowest log loss

The model with the lowest test-set log loss was **gradient boosting (GBDT)** with **log loss = 0.4127**. This is slightly better than logistic regression's log loss of *0.4152* and

noticeably better than bagging (*0.4306*), stacking (*0.4284*), random forest (*0.4717*), and AdaBoost (*0.5102*).

Q3. Churn-class recall improvements

No ensemble model produced a meaningful improvement in recall for the churn class beyond the best single model. The highest churn recall was **0.5241**, achieved by both **logistic regression** and **gradient boosting (GBDT)**. Other ensemble methods performed worse: stacking (*0.4973*), AdaBoost (*0.5053*), random forest (*0.4759*), and bagging (*0.4064*).

Q4. Stacking versus the best single model

Stacking did not outperform the best single model in this case. Compared to logistic regression, stacking had slightly lower accuracy (*0.7949 vs 0.7970*), worse probability quality (*log loss 0.4284 vs 0.4152*), and lower churn recall (*0.4973 vs 0.5241*). This illustrates that stacking is not guaranteed to improve performance when the base learners already capture most of the predictive signal.

Q5. Deployment recommendation

In a real telecom retention system, **gradient boosting (GBDT)** would be the most defensible choice. It achieved the lowest log loss (*0.4127*), tied for the highest churn recall (*0.5241*), and delivered competitive accuracy (*0.7942*). Because retention actions are typically driven by risk thresholds and expected value calculations, well-calibrated probabilities are more important than small differences in accuracy, making gradient boosting particularly suitable for operational deployment.

Case #2: Employee Attrition (Ensemble Models)
This case study uses the **Employee Attrition** dataset (*Employee_Attrition.csv*) to evaluate whether ensemble learning methods improve prediction quality over single models. Your goal is to build and compare multiple **ensemble classifiers** that estimate whether an employee will leave the company (*Attrition* = Yes/No), and to analyze

tradeoffs in accuracy, probability calibration, recall for high-risk employees, and computational cost.

Dataset attribution: This dataset is widely distributed as an "Employee Attrition / HR Analytics" teaching dataset based on IBM HR sample data and is provided in this course as *Employee_Attrition.csv*. See details on Kaggle.com The Employee Attrition dataset is available in the prior chapter if you need to reload it.

Prediction goal: Predict whether *Attrition* is Yes (employee leaves) or No (employee stays) using the remaining columns as predictors. Use a supervised learning workflow with preprocessing pipelines, a stratified train/test split, and evaluation on a holdout test set.

For reproducibility, use **random_state = 27** everywhere that a random seed is accepted.

Tasks

- Inspect the dataset: number of rows and columns, data types, missing values, and the class distribution of *Attrition*.
- Define *X* and *y*, where *y = Attrition*. Remove identifier-style columns (for example *EmployeeNumber*) if present.
- Create an 80/20 train/test split using *random_state=27* and *stratify=y*.
- Build a preprocessing pipeline using *ColumnTransformer*: scale numeric features with *StandardScaler* and one-hot encode categorical features using *OneHotEncoder(handle_unknown="ignore")*.
- Train a baseline *logistic regression* model and record test-set accuracy, log loss, ROC AUC, and recall for the attrition class.
- Train the following ensemble models inside the same preprocessing pipeline:
  - Bagging classifier (shallow decision trees)
  - Random forest classifier
  - AdaBoost (decision stumps)
  - Gradient boosting (GBDT)
  - Stacking (logistic regression + random forest + k-NN → logistic regression)

- Evaluate each model using: accuracy, precision, recall, F1-score, ROC AUC, and log loss. Store all results in a single comparison table.
- Plot confusion matrices for logistic regression, gradient boosting, and stacking. Compare how many high-risk employees (Attrition = Yes) are missed by each model.
- Compare training time and model complexity across all methods. Identify which models would be difficult to deploy in a real-time HR analytics system.
- Select one model as the recommended production system for predicting employee attrition. Justify your choice using probability quality, recall, interpretability, and operational constraints.

Analytical questions

1. How many rows and columns are in the Employee Attrition dataset, and what percentage of employees have *Attrition = Yes*?
2. Which ensemble model achieved the highest test-set accuracy? Which achieved the lowest log loss?
3. Which model achieved the highest recall for employees who actually left the company? Why is this metric especially important for HR planning?
4. Compare gradient boosting and random forests in terms of probability calibration (log loss) and training time. Which appears more suitable for operational forecasting?
5. Did stacking meaningfully outperform the best single model? Support your answer using at least two quantitative metrics.
6. Which features were most important in the tree-based ensemble models, and do they align with HR intuition about employee turnover?
7. Reflection (4–6 sentences): Discuss how ensemble learning changes the bias–variance tradeoff compared to a single decision tree in this dataset. Would you expect similar improvements on much smaller datasets? Why or why not?

Employee Attrition Ensemble Case Answers

These answers assume you used the provided Employee Attrition dataset, removed ID-like columns (such as EmployeeNumber), created an 80/20 stratified train/test split using **random_state = 27**, and evaluated all models on the same holdout test set. Results are reported exactly as shown in your experiment output.

Q1. Dataset size

The dataset contains **1,470 rows** and **35 columns**.

Q2. Attrition rate

The attrition rate (share of employees with *Attrition = Yes*) is **0.1612**, corresponding to **237 attritions out of 1,470 employees** (about **16.12%**).

Q3. Baseline and data split context

After preprocessing and removing ID-like fields, the feature matrix contained **30 predictors**.

The stratified split produced **1,176 training rows** and **294 test rows**.

Q4. Logistic regression performance

Logistic regression achieved test-set **accuracy = 0.8878** and **log loss = 0.3511**, with *ROC AUC = 0.7991* and *macro F1 = 0.7587*.

For the attrition class (*Yes*), precision was **0.7188** and recall was **0.4894**, meaning the model correctly identified about 49% of employees who eventually left.

This model also trained extremely quickly (*0.025 seconds*), making it a strong baseline in both performance and operational efficiency.

Q5. Ensemble model comparison (accuracy and log loss)

Among all ensemble models evaluated (bagging, random forest, AdaBoost, gradient boosting, and stacking), the results varied substantially across accuracy, probability quality, and training cost.

The highest overall accuracy was achieved by **logistic regression (0.8878)**, narrowly outperforming stacking (0.8844) and gradient boosting (0.8810).

The lowest log loss was achieved by the **stacking model (0.3207)**, indicating the best-calibrated probability estimates among all models tested.

Q6. Attrition recall comparison

Recall for the attrition class varied widely across models:

- Logistic regression: **0.4894**
- Stacking: *0.4468*
- Gradient boosting: *0.3404*
- AdaBoost: *0.2979*
- Random forest: *0.1702*
- Bagging: *0.1064*

Logistic regression detected the largest share of actual attrition cases, while bagging and random forests were extremely conservative and missed most departing employees.

Q7. Feature importance insights

Both random forest and gradient boosting highlighted similar dominant predictors of attrition risk.

Top features included:

- MonthlyIncome
- Age
- DistanceFromHome

- TotalWorkingYears
- YearsAtCompany
- OverTime (Yes/No)
- JobSatisfaction
- StockOptionLevel

These results suggest that compensation, career stage, workload, commute burden, and job satisfaction all play major roles in employee retention.

Q8. Stacking versus the best single model

The best single model by log loss was logistic regression (*log loss = 0.3511*, accuracy = 0.8878).

The stacking ensemble improved probability quality substantially (*log loss = 0.3207*) while achieving nearly identical accuracy (*0.8844*).

However, stacking required **over 12 seconds** of training time, compared to *0.025 seconds* for logistic regression.

Q9. Deployment recommendation

If the primary objective is **operational simplicity, interpretability, and fast retraining**, logistic regression is the most practical choice. It achieves the highest accuracy, the highest attrition recall, strong probability quality, and negligible training cost.

If the organization instead prioritizes **probability calibration for ranking employees by risk** (for example, allocating retention incentives to the top 5–10% most at-risk employees), the stacking model is preferable despite its computational cost.

In real HR systems, the final choice should reflect business costs: missing a true attrition case (false negative) may be more expensive than contacting an employee who would have stayed anyway (false positive).

Q10. Reflection (sample answer)

This case illustrates that ensemble methods do not automatically dominate simpler models. While stacking improved probability calibration, it did not materially improve accuracy or recall beyond logistic regression. Ensemble models are most valuable when nonlinear interactions dominate or when probability ranking quality is critical. In structured HR data with strong linear signals, simpler models can remain highly competitive while being easier to deploy and explain.

Case #3: Telco Support Ticket Priority Dataset (Ensembles)
This case uses a Telco customer support dataset of service tickets. Your goal is to build and compare **ensemble-based multiclass classification models** that predict ticket *priority* (Low, Medium, High) using structured ticket and customer context variables. You will evaluate models using both *accuracy* (threshold-based) and *multiclass log loss* (probability-based).

Dataset attribution: The dataset file for this case is *Support_tickets.csv*. See details on Kaggle.com The Support Ticket Priority dataset is available in the prior chapter if you need to reload it.

Prediction goal: Predict *priority* (Low, Medium, High). Treat the classes as *nominal* (not ordered) and use standard multiclass classification metrics.

Recommended feature set: Use a mix of numeric and categorical predictors. Prefer the human-readable categorical columns (for example: *day_of_week*, *company_size*, *industry*, *customer_tier*, *region*, *product_area*, *booking_channel*, *reported_by_role*, *customer_sentiment*) and numeric operational columns (for example: *org_users*, *past_30d_tickets*, *past_90d_incidents*, *customers_affected*, *error_rate_pct*, *downtime_min*, plus binary flags such as *payment_impact_flag* and *security_incident_flag*). Exclude identifier-like columns such as *ticket_id*.

Note on duplicate encodings: This dataset includes both readable categorical columns and numeric-coded versions (for example, *industry* and *industry_cat*). Use only one representation. The recommended approach is to use the readable categorical columns with one-hot encoding.

For reproducibility, use **random_state = 27** everywhere a random seed is accepted.

Tasks

- Inspect the dataset: report the number of rows and columns, list the unique values of *priority*, and compute class counts and percentages.
- Create *X* and *y* where *y = priority*. Remove *ticket_id* and all *_cat* columns.
- Split the data into training and test sets (80/20) using *random_state=27* and *stratify=y*.
- Build a preprocessing pipeline using *StandardScaler* for numeric predictors and *OneHotEncoder(handle_unknown="ignore", sparse_output=False)* for categorical predictors.
- Train a baseline classifier that always predicts the most frequent priority class. Report test-set accuracy and multiclass log loss using class proportions as constant probabilities.
- Train and evaluate a multiclass *RandomForestClassifier*. Report accuracy, macro F1, classification report, and log loss.
- Train and evaluate a *BaggingClassifier* with shallow decision trees as base learners. Use at least 100 estimators and report the same metrics.
- Train and evaluate a *GradientBoostingClassifier* (GBDT). Report accuracy, macro F1, and log loss.
- Train and evaluate an *AdaBoostClassifier* using shallow trees (stumps). Report the same metrics.
- Train a *StackingClassifier* with base learners: logistic regression, random forest, and k-NN, and a logistic regression meta-model. Report test metrics and training time.
- Create a single comparison table including all models (baseline, bagging, random forest, AdaBoost, gradient boosting, stacking) with accuracy, macro F1, and log loss.
- Create two bar charts: one for accuracy and one for log loss across all models.
- Create confusion matrices for the two best-performing models (based on log loss and macro F1) and interpret the most common misclassifications.
- Write a short deployment checklist (5–8 bullets) describing how you would choose a model in a real support organization, considering probability calibration, minority-

class behavior, interpretability, and runtime constraints.

Analytical questions

1. How many rows and columns are in the dataset, and what are the class proportions for Low, Medium, and High priority?
2. How does the baseline model perform in terms of accuracy and log loss, and why is log loss especially informative in multiclass settings?
3. Which ensemble method achieved the lowest multiclass log loss?
4. Which model achieved the highest macro-average F1 score?
5. Which priority class is hardest to predict correctly across models?
6. Compare bagging and random forests. Which performs better and why?
7. Did boosting improve probability calibration relative to bagging?
8. Did stacking improve performance over the best single ensemble method?
9. Which two priority levels are most frequently confused in the confusion matrices?
10. Short reflection (5–8 sentences): Which model would you deploy in a real support operation and why? Discuss the cost of misclassifying High-priority tickets, interpretability tradeoffs, and probability quality.

Telco Support Ticket Priority Case Answers

These answers assume you used the Support Tickets dataset, removed identifier and duplicate-coded *_cat* columns, created an 80/20 stratified train/test split with **random_state = 27**, and evaluated all models on the same holdout test set. The target label *priority* is multiclass (High, Medium, Low) and metrics include *accuracy*, *macro F1*, and *multiclass log loss*.

Q1. Dataset size

The dataset contains **50,000 rows** and **33 columns** before preprocessing.

Q2. Class distribution

The target has three classes: **high**, **low**, and **medium**.

Class counts are: **low = 25,000**, **medium = 17,500**, and **high = 7,500**.

Class proportions are: **low = 0.50**, **medium = 0.35**, and **high = 0.15**. This imbalance matters because a model can appear strong on accuracy by doing well on the majority class (*low*) while under-serving the *high* class.

Q3. Baseline model

The baseline predicts the most frequent class (*low*) for every ticket, which yields test-set **accuracy = 0.5000**.

Using the training-set class proportions as constant probabilities for every ticket, the baseline multiclass **log loss = 0.9986**. This is a useful reference point for probability quality: any practical model should reduce log loss substantially below this value.

Q4. Logistic regression results

Multiclass logistic regression achieves **accuracy = 0.8648**, **macro F1 = 0.8469**, and **log loss = 0.3406**.

From the classification report, the lowest recall is for the **high** class (*recall = 0.785*). Operationally, this matters because missed *high*-priority tickets are typically more costly than confusing *low* and *medium*.

Q5. Bagging results (depth = 3)

Bagging with shallow trees performs worse than the stronger ensembles in this run: **accuracy = 0.7841**, **macro F1 = 0.7479**, and **log loss = 0.4883**.

The largest weakness is on the *high* class (*recall = 0.545*), indicating many urgent tickets are not being flagged as *high* at the default decision rule.

Q6. Best depth by log loss and accuracy

In your ensemble-focused run, the strongest probability performance comes from boosting and stacking rather than tuning a single shallow tree depth. In the results you reported, the best model by log loss is **Stacking** (log loss **0.1653**), and the best model by accuracy is **Gradient boosting (GBDT)** (accuracy **0.9394**). This illustrates why "best" can differ by metric: log loss rewards well-calibrated probabilities, while accuracy rewards correct hard labels.

Q7. Feature importance and interpretation

Feature importance depends on the specific model you extract it from (trees/forests/GBDT provide impurity-based importances, while logistic regression often uses coefficients). A practical business interpretation approach is: identify the top drivers for predicting *high* priority (for example, operational severity signals such as *downtime_min*, *customers_affected*, *error_rate_pct*, and incident flags) and confirm they align with how the support team triages tickets. The key decision implication is whether the model's most important predictors are "actionable" (can be improved upstream) and "trustworthy" (not simply proxies for data entry artifacts).

Q8. Confusion matrices for the two best models

For **Stacking**, the confusion matrix shows the most common confusions are between **high ↔ medium** and **medium ↔ low**, with very little direct confusion between *high* and *low*. Concretely, stacking misclassifies *high* as *medium* **145** times (out of 1,500 high tickets), and misclassifies *medium* as *low* **205** times (out of 3,500 medium tickets).

For **Gradient boosting (GBDT)**, the same pattern holds: most errors are "adjacent" (high vs medium, medium vs low), not extreme. GBDT misclassifies *high* as *medium* **205** times, and misclassifies *medium* as *low* **185** times. In a ticket triage setting, these are often less damaging than confusing *high* with *low*, but they still affect staffing and response times.

Q9. Best model by log loss and by macro F1

Best by probability quality (lowest log loss) is **Stacking (LR + RF + kNN → LR)** with **log loss = 0.1653**, *macro F1 = 0.9196*, and *accuracy = 0.9280*.

Best by balanced classification performance (highest macro F1) is **Gradient boosting (GBDT)** with **macro F1 = 0.9300**, *accuracy = 0.9394*, and *log loss = 0.2047*.

Random forest is the next-closest strong option (*accuracy = 0.9205*, *macro F1 = 0.9092*, *log loss = 0.2727*) and trains much faster than stacking/GBDT in your run.

Q10. Reflection sample answer

In a real support operation, I would start with **stacking** if probability quality drives decisions (for example, queue ranking, staffing forecasts, or cost-sensitive thresholding), because it achieved the best *log loss* (0.1653) while still maintaining high accuracy (0.9280) and strong macro F1 (0.9196). If the priority assignment is primarily used as a hard label and the goal is to maximize correct classification across classes, I would choose **gradient boosting** because it produced the best overall accuracy (0.9394) and macro F1 (0.9300). In both cases, I would pay special attention to the business cost of misrouting *high* tickets; the confusion matrices suggest most errors are between adjacent levels (high vs medium, medium vs low), which is preferable to confusing high with low. Finally, deployment constraints may justify **random forest** as a strong compromise because it performs well and trains far faster than stacking or GBDT in this run, which can matter for frequent retraining or large-scale production pipelines.

14.12Learning Objectives

<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to explain how ensemble methods reduce prediction error through variance reduction (bagging) and bias reduction (boosting)
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to build and tune random forest models with appropriate hyperparameters for classification and regression tasks
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to implement gradient boosting classifiers (including XGBoost) and understand how sequential learners correct prior errors
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to construct stacking ensembles that combine multiple base learners with a meta-learner

for improved performance <{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to compare single models to ensemble approaches using cross-validation and select the best method for a given problem

14.13Assignment

Complete the assignment below:

This assessment can be taken online.